COURS PAR CORRESPONDANCE

ANALYSTE EN INFORMATIQUE

Système d'exploitation

Module

Bureautiques & architectures des ordinateurs

Système d’exploitation

(Lecture recommandée)

SOMMAIRE

1. LE SYSTEME D'EXPLOITATION 4

1.1 GENERALITES 4
1.1.1 OBSERVATION PRATIQUE 4
1.1.2 CHARGEMENT DU SYSTEME D'EXPLOITATION 4
1.1.3 ROLE D'UN SYSTEME D'EXPLOITATION 4
1.2 FONCTIONNALITES DE BASE D'UN SYSTEME D'EXPLOITATION 5
1.2.1 INTERFACE HOMME MACHINE (ENVIRONNEMENT UTILISATEUR) 5
1.2.2 GESTION DES ENTREES-SORTIES ET DES INTERRUPTIONS 5
1.2.3 GESTION DES RESSOURCES ET DES TRAVAUX 5
1.2.4 GESTION DE LA MEMOIRE 5
1.2.5 GESTION DES FICHIERS 5
1.2.6 TRAITEMENT DES PROGRAMMES D'APPLICATION 5
1.2.7 GESTION DES ERREURS 6
1.3 LES DIFFERENTES COUCHES D'UN SYSTEME D'EXPLOITATION 6
1.3.1 DECOMPOSITION GLOBALE D'UN SYSTEME INFORMATIQUE 6
1.3.2 DECOMPOSITION DU SYSTEME D'EXPLOITATION 7
1.3.3 LE PROGRAMME D'APPLICATION UTILISE LES PRIMITIVES DU SE POUR DEUX RAISONS 8
1.4 LE BIOS (BASIC INPUT OUTPUT SYSTEM) 8
1.5 CLASSIFICATION DES SYSTEMES D'EXPLOITATION 10
1.5.1 CLASSIFICATION PAR MODE DE TRAITEMENT (MONO, MULTI-UTILISATEUR) 10
1.5.2 CLASSEMENT PAR MODE DE PROGRAMMATION (MONO, MULTI-TACHE) 11
1.5.3 SYSTEMES D'EXPLOITATION OUVERTS OU FERMES 13

2. LA GESTION DES ENTREES-SORTIES ET DES INTERRUPTIONS 14

2.1 LES ENTREES-SORTIES 14
2.1.1 PROBLEMATIQUE DES ENTREES-SORTIES 14
2.1.2 NOTION D'INTERFACE 15
2.1.3 LES DIFFERENTS TYPES DE LIAISONS 19
2.2 COMPLEMENT SUR LES ENTREES-SORTIES 19
2.2.1 SIMULTANEITE ENTRE TRAITEMENT ET ENTREES-SORTIES 19
2.2.2 L'ACCES DIRECT A LA MEMOIRE (DMA) 20
2.2.3 LES CANAUX D'ENTREES-SORTIES 21
2.3 LES INTERRUPTIONS 21
2.3.1 POURQUOI DES INTERRUPTIONS ? 21
2.3.2 MECANISME D'INTERRUPTION 22
2.3.3 DEROULEMENT D'UNE INTERRUPTION 22
2.3.4 CLASSIFICATION DES INTERRUPTIONS 24
2.3.5 LES INTERRUPTIONS LOGICIELLES 24
2.3.6 EXEMPLES 25
2.3.7 LA PRIORITE DES INTERRUPTIONS 25
2.3.8 EXEMPLE : LES INTERRUPTIONS SOUS MS-DOS (CAS DU P.C. AT) 26

3. LA GESTION DE LA MEMOIRE 30

3.1 POURQUOI UNE GESTION DE LA MEMOIRE ? 30
3.1.1 SIMULATION D'UNE MEMOIRE INFINIE 30
3.1.2 TRANSPARENCE VIS A VIS DES PROCESSUS 30
3.1.3 PROTECTION DES PROCESSUS 30

3.1.4 GESTION DES PARTIES DE PROGRAMME 30
3.1.5 PARTAGE DE CODE 30
3.2 DEFINITIONS 31
3.2.1 MEMOIRE PHYSIQUE 31
3.2.2 MEMOIRE VIRTUELLE 31
3.2.3 LE SWAPPING 31
3.3 LA SEGMENTATION 31
3.3.1 PRINCIPES 31
3.3.2 CHARGEMENT DES SEGMENTS EN MEMOIRE CENTRALE ET ADRESSAGE 33
3.3.3 AVANTAGES ET INCONVENIENTS DE LA SEGMENTATION 33
3.4 LA PAGINATION 35
3.4.1 PRINCIPE ET MISE EN OEUVRE 35
3.4.2 EXEMPLE 35
3.4.3 CHOIX DE LA TAILLE DES PAGES 36
3.5 STRATEGIES DE REMPLACEMENT 37
3.5.1 REMPLACEMENT DES SEGMENTS 37
3.5.2 REMPLACEMENT DES PAGES 37

4. LA GESTION DES TRAVAUX (PROCESSUS) ET DES RESSOURCES 39

4.1 DEFINITIONS 39
4.1.1 PROCESSEUR 39
4.1.2 PROCESSUS 39
4.2 MODES DE FONCTIONNEMENT (RAPPEL) 39
4.2.1 MONOPROGRAMMATION 39
4.2.2 MULTIPROGRAMMATION 39
4.2.3 MULTITRAITEMENT 39
4.3 ETAT DES PROCESSUS 39
4.4 ALLOCATION DE L'UNITE CENTRALE 40
4.4.1 NOTION D'ORDONNANCEMENT (SCHEDULING) 40
4.4.2 NOTION DE PRIORITE 40
4.4.3 NOTION DE QUANTUM 41
4.4.4 DIFFERENTES STRATEGIES D'ORDONNACEMENT 41

5. LA GESTION DE L'ENVIRONNEMENT ET DES ERREURS 44

5.1 L'INTERPRETEUR DE COMMANDES (SHELL : ENVIRONNEMENT DE TRAVAIL) 44
5.1.1 ROLE ET FONCTIONS 44
5.1.2 LES DEUX TYPES DE COMMANDES 44
5.1.3 PRINCIPES DU DIALOGUE 44
5.2 LA GESTION DE L'ESPACE DISQUE 44
5.2.1 NOTION DE FICHIER 45
5.2.2 NOTION D'UNITE LOGIQUE 45
5.2.3 NOTION DE SOUS-REPERTOIRE 45
5.2.4 NOTION DE CHEMIN D'ACCES A UN FICHIER 46
5.3 GESTION DES INCIDENTS 46
5.3.1 PRINCIPAUX INCIDENTS 46
5.3.2 REACTION AUX INCIDENTS 47
5.3.3 MAINTENANCE DES MEMOIRES DE MASSE 47

6. LA GESTION DES FICHIERS 49

6.1 LE SYSTEME DE GESTION DE FICHIERS DE MS-DOS SUR UNE DISQUETTE 3,5'' 49
6.1.1 LE SYSTEME DE GESTION DE FICHIERS 49
6.1.2 COMPOSITION DU REPERTOIRE RACINE (DIRECTORY) 51
6.1.3 LA FAT 52
6.1.4 LECTURE D'UN FICHIER SUR LA DISQUETTE 52
6.2 LA STRUCTURE DE L'INFORMATION 52
6.2.1 EXEMPLES 53
6.2.2 CODAGE 53
6.2.3 DESCRIPTION D'UNE REALITE : LE FICHIER 53
6.3 ORGANISATION LOGIQUE A STRUCTURE FIXE 54
6.3.1 ORGANISATION LOGIQUE STRUCTUREE 54
6.3.2 NOTION D'ENREGISTREMENT 54
6.3.3 ACCES A L'INFORMATION 54
6.4 ORGANISATION LOGIQUE A STRUCTURE VARIABLE 56
6.5 ENREGISTREMENT LOGIQUE, ENREGISTREMENT PHYSIQUE 56
6.5.1 ENREGISTREMENT LOGIQUE 57
6.5.2 ENREGISTREMENT PHYSIQUE 57
6.6 ORGANISATION PHYSIQUE 57
6.6.1 ORGANISATION PHYSIQUE SEQUENTIELLE 57
6.6.2 ORGANISATION PHYSIQUE ADRESSEE 57
6.7 EN RESUME 58

7. LA GESTION DES PROGRAMMES 59

7.1 LE LANGAGE MACHINE 59
7.2 LE LANGAGE D'ASSEMBLAGE 59
7.3 LES LANGAGES DE PROGRAMMATION EVOLUES 61
7.3.1 LES DIFFERENTES ETAPES 62
7.3.2 FONCTIONNEMENT DE LA COMPILATION 63
7.3.3 FONCTIONNEMENT DE L'EDITEUR DE LIEN (LINK) 64
7.3.4 L'INTERPRETEUR 65
7.4 CLASSIFICATION DES LANGAGES 65
7.5 LES LANGAGES LES PLUS CONNUS 67

1. LE SYSTEME D'EXPLOITATION

1.1 GENERALITES

1.1.1 OBSERVATION PRATIQUE

Démarrons un P.C. sans disque dur. Après quelques tests, la machine demande d'insérer une disquette ("disquette système"). Sans cette disquette, le P.C. reste bloqué, il ne sait pas dialoguer avec l'utilisateur. Il manque le système d'exploitation qui est nécessaire au fonctionnement d'un ordinateur.

1.1.2 CHARGEMENT DU SYSTEME D'EXPLOITATION

Cette opération est appelée "boot strap" (chausse-pied) ou bien "boot". Sur certaines machines, le système est entièrement contenu en ROM et s'initialise dès la mise sous tension.

Sur les autres, en particulier sur les P.C., seule une toute petite partie est présente en ROM (appelée le BIOS). Le BIOS est tout d'abord initialisé, puis, le reste du système est chargé en mémoire centrale à partir d'une disquette système (bootable) ou du disque dur. Le système est ensuite lancé.

1.1.3 ROLE D'UN SYSTEME D'EXPLOITATION

Le système d'exploitation gère les ressources matérielles et logicielles de la machine sur demande du programme ou de l'utilisateur directement.

Les fonctions du système d'exploitation sont très variées, néanmoins on peut classer ses diverses fonctionnalités en deux catégories :

Celles qui sont mises à la disposition de l'utilisateur pour qu'il puisse adapter son environnement de travail, créer ou effacer des fichiers, préparer des supports de stockage d'information (formater), ...

On appelle cette catégorie les commandes utilisateurs ou ***interface*** de dialogue entre l'utilisateur et la machine. Elles permettent de lancer des opérations complexes avec des instructions simples.

Celles qui permettent le fonctionnement du système informatique. Ces fonctionnalités sont pratiquement inaccessibles à l'utilisateur. Dans ce domaine, le système d'exploitation assure :

- La gestion de l'environnement,
- La gestion des entrées sorties,
- La gestion des interruptions,
- La gestion des ressources,
- La gestion des travaux,
- La gestion de la mémoire,
- La gestion des fichiers,
- La gestion des programmes,
- La gestion des erreurs.

1.2 FONCTIONNALITES DE BASE D'UN SYSTEME D'EXPLOITATION

1.2.1 INTERFACE HOMME MACHINE (ENVIRONNEMENT UTILISATEUR)

Le système d'exploitation doit permettre à l'utilisateur de dialoguer avec la machine par un ensemble de commandes simples (voir précédemment) ou même de manière graphique (surcouche du système).

1.2.2 GESTION DES ENTREES-SORTIES ET DES INTERRUPTIONS

Le système d'exploitation s'occupe du dialogue avec les périphériques (échange de données).

1.2.3 GESTION DES RESSOURCES ET DES TRAVAUX

Le système d'exploitation doit fournir des outils permettant à plusieurs processus de se synchroniser entre eux et de pouvoir s'exécuter dans l'unité centrale en s'imbriquant.

1.2.4 GESTION DE LA MEMOIRE

Il gère la mémoire centrale afin d'allouer la quantité nécessaire aux programmes qui en demandent. Il doit ainsi optimiser au maximum l'espace mémoire disponible.

1.2.5 GESTION DES FICHIERS

Le système d'exploitation doit organiser à tout moment la structure des supports de stockage. C'est lui qui est responsable de la gestion des fichiers sur disque et de leur cohérence.

1.2.6 TRAITEMENT DES PROGRAMMES D'APPLICATION

C'est le système qui doit assurer le chargement et l'exécution correcte des programmes d'application.

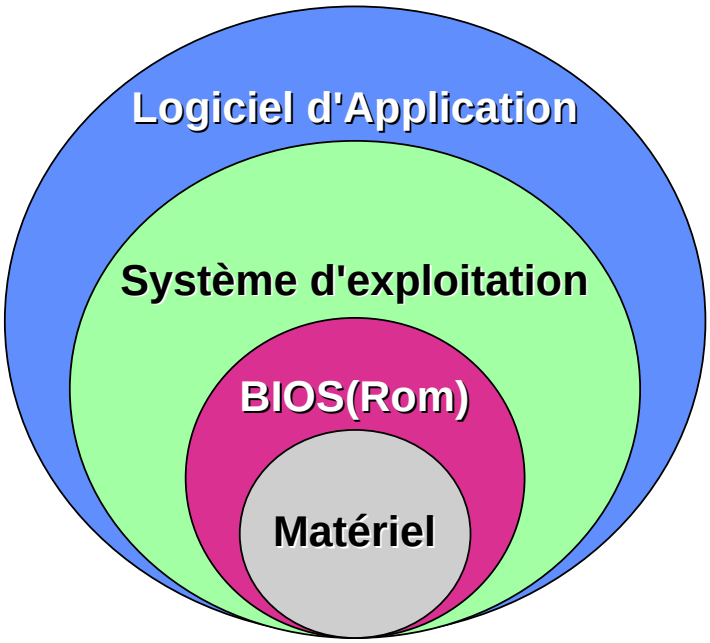
Il peut également (mais ce n'est pas systématique) mettre à disposition des outils (compilateurs, assembleurs, éditeurs de liens) permettant au programmeur de réaliser ses propres logiciels exécutables.

1.2.7 GESTION DES ERREURS

La gestion des erreurs susceptibles de se produire pendant l'exécution d'une opération revient également au système d'exploitation. Il doit ou bien essayer de résoudre le problème, ou bien le signaler soit à l'utilisateur par le biais de messages d'erreurs, soit au programme en cours d'exécution grâce aux codes d'erreurs.

1.3 LES DIFFERENTES COUCHES D'UN SYSTEME D'EXPLOITATION

1.3.1 DECOMPOSITION GLOBALE D'UN SYSTEME INFORMATIQUE



1.3.1.1 MATERIEL

L'ensemble des éléments composant la machine et les périphériques.

1.3.1.2 BIOS

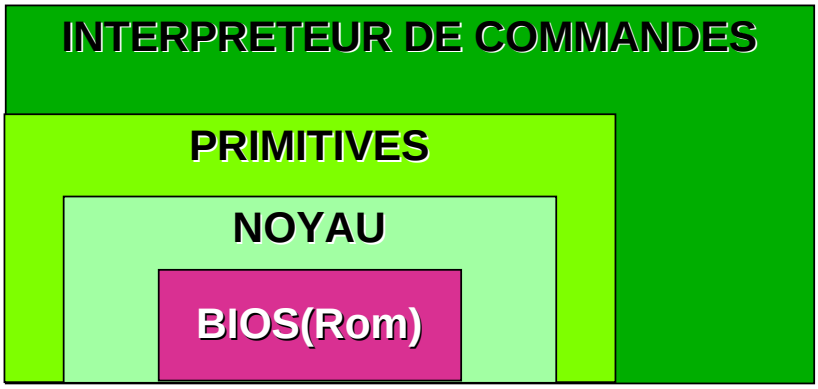
Le BIOS (Basic Input Output System) permet à un système d'exploitation standard de fonctionner sur des machines dont les caractéristiques internes peuvent varier. Le BIOS contient des petits programmes permettant à la machine de communiquer avec l'extérieur (unité centrale / périphériques standards). Par exemple, la réception d'un caractère du clavier.

1.3.1.3 SYSTEME D'EXPLOITATION

Le système d'exploitation regroupe l'ensemble des primitives (petits programmes) exécutant chacune une tâche précise. Ces primitives sont appelées par l'utilisateur ou le logiciel d'application

Exemple : affichage à l'écran d'un caractère saisi au clavier.

1.3.2 DECOMPOSITION DU SYSTEME D'EXPLOITATION



Couche système = Interpréteur + (Primitives + Noyau)

1.3.2.1 NOYAU

Le noyau, cœur du système correspond à l'ensemble des sous-programmes constituant le système d'exploitation. Cette partie correspond aux fichiers MSDOS.SYS et IO.SYS de MS/DOS.

Sous UNIX, le noyau assure les fonctions suivantes (entre autres) :

- Gestion des données du système (informations sur les éléments de la machine) ;
- Protection du système et des données (le système est chargé en mémoire dans une zone réservée) ;
- Transfert de l'information entre les différents éléments de la machine ;
- Fonctionnement Multi-Utilisateurs en gérant l'exploitation du processeur ;
- Gestion de la mémoire centrale.

1.3.2.2 PRIMITIVES

Les primitives constituent l'ensemble (liste) des sous-programmes d'appel au système, l'ensemble des points d'entrée dans le noyau. La primitive Write () d'UNIX permet de lancer le sous - programme d'écriture disque. Ces primitives sont à la disposition de l'utilisateur et du programme d'application. Le programme fera appel aux primitives pour réaliser des opérations déjà programmées dans le système d'exploitation et souvent exécutables uniquement par le système.

1.3.2.3 INTERPRETEUR DE COMMANDES

L'interpréteur de commandes (Shell sous UNIX) vérifie les commandes entrées par l'utilisateur, les déchiffre et les exécute si elles sont correctes. Il correspond au fichier COMMAND.COM de MS/DOS. Il donne aussi la possibilité à l'utilisateur de créer lui-même ses propres commandes.

1.3.3 LE PROGRAMME D'APPLICATION UTILISE LES PRIMITIVES DU SE POUR DEUX RAISONS

1.3.3.1 PREMIERE RAISON

Il est inutile de demander au programmeur de reprogrammer certaines opérations de base, communes à tous les programmes.

Exemple : Une écriture disque, la lecture d'un caractère au clavier. En effet, dans le cas de l'écriture disque, il faut : s'assurer que le lecteur est prêt, choisir la bonne vitesse de rotation du disque, déplacer les têtes de lecture, ralentir la vitesse de rotation pour écrire la donnée, etc...

Ces instructions de base, de très bas niveau, très proche de l'électronique et de la mécanique de la machine sont confiées au système d'exploitation et au BIOS. Le programmeur sous-traitera son écriture disque au système d'exploitation, qui fera appel à des routines du BIOS. Le programmeur pourra alors se consacrer à une programmation plus riche en abstraction, sans tout réinventer à chaque fois.

En réalité, le programme applicatif fait très souvent appel au système d'exploitation. Plus le langage sera évolué (proche de l'utilisateur) et moins ces appels seront visibles pour le programmeur.

1.3.3.2 DEUXIEME RAISON

Le système d'exploitation exécute des opérations de très bas niveau, très proches de la machine. Les erreurs de programmation à ce niveau peuvent être lourdes de conséquence. Afin d'éviter le pire, certaines instructions du jeu d'instruction de la machine ne pourront être utilisées que par le système d'exploitation. La primitive pourra composer avec toutes les instructions de base de la machine. Une fois programmé correctement, le système d'exploitation ne risque plus de représenter un danger potentiel pour la machine.

On dit alors que le système d'exploitation utilise le microprocesseur en mode maître ou superviseur, il a tous les droits.

Parfois, pour certaines opérations, le programmeur doit obligatoirement s'adresser au système d'exploitation. Le programmeur utilise le microprocesseur en mode utilisateur ou mode esclave.

1.4 LE BIOS (BASIC INPUT OUTPUT SYSTEM)

Il est difficile d'appréhender le système d'exploitation sans définir ce qui le relie à la machine: le BIOS.

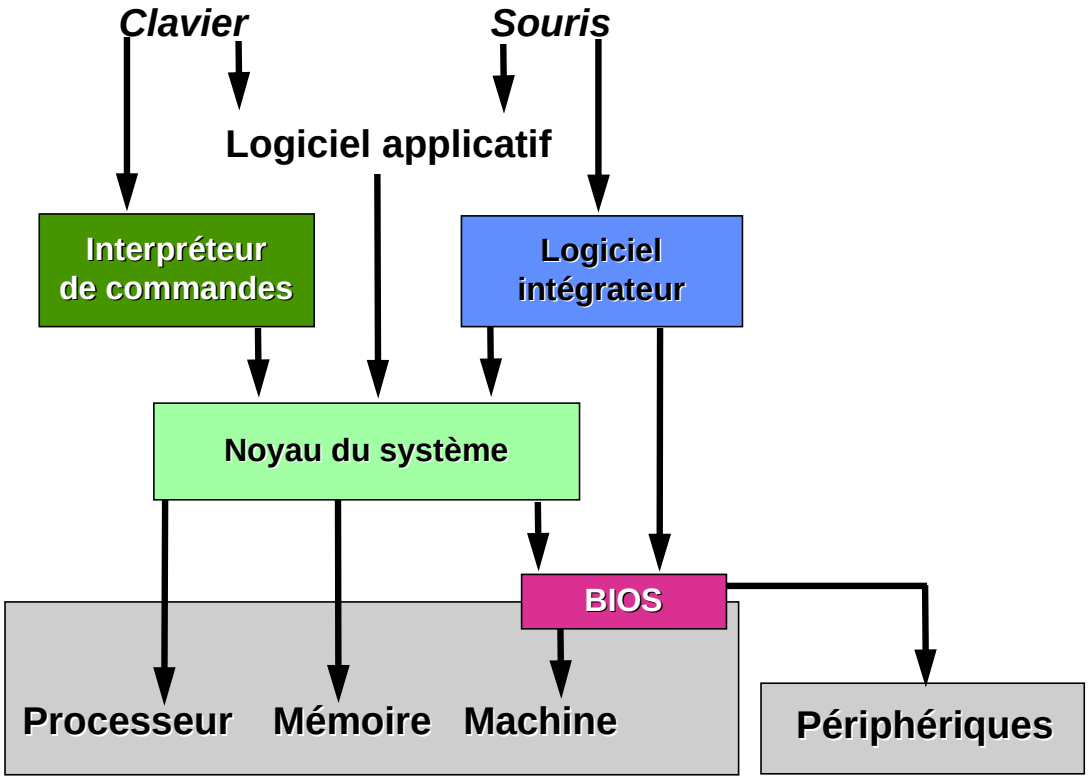
Comme le montre le schéma suivant, ce n'est pas le système d'exploitation qui donne les ordres à la machine mais un programme spécial. Ce programme, de petite taille, appelé BIOS, contient les différents programmes nécessaires aux tests des différents éléments de l'ordinateur, au démarrage du disque dur ainsi qu'au chargement du système d'exploitation.

Le BIOS contient également des ***fonctions*** (sous-programmes) permettant au système d'exploitation de dialoguer et de commander les périphériques standard (clavier, écran, disque dur, lecteur de disquette) en mode standard (utilisation d'une imprimante ou de l'écran en mode texte).

Le BIOS comporte aussi un module ***setup*** permettant de configurer la partie matérielle du système.

C'est le BIOS qui effectue les transferts de données avec les périphériques standards. Par contre***, la gestion du processeur et de la mémoire centrale est du ressort du système d'exploitation***.

Le BIOS est une spécificité du monde P.C.



Le système d'exploitation communique au BIOS, sous forme de messages, les différentes commandes que celui-ci doit faire exécuter aux périphériques. Ces messages sont traduits en commandes BIOS. Une primitive du système peut donc se décomposer en plusieurs commandes BIOS.

Dans le monde P.C., le BIOS est un ensemble de procédures standard contrôlant les entrées-sorties. Le BIOS est implanté dans la ROM de la carte mère.

Dans d'autres systèmes d'exploitation, le BIOS (au moins son équivalent) fait partie du système : c'est la couche la plus basse qui devra être modifiée pour utiliser le système sur des machines différentes.

Compatibilité et BIOS :

Le BIOS est vendu avec la machine. Son action est adaptée au processeur utilisé. Par contre, le fait qu'un BIOS soit installé sur une carte mère ne présuppose en rien quel sera le système d'exploitation utilisé. Un même BIOS peut utiliser plusieurs systèmes d'exploitations différents. Lorsque des BIOS différents réalisent pratiquement les mêmes actions, on dit qu'ils sont compatibles. C'est donc au niveau du BIOS que se juge la compatibilité d'une machine.

Evolutivité et BIOS :

Il est aujourd'hui possible de modifier certains paramètres du BIOS car ceux-ci sont écrits dans une mémoire flash ou une EPROM (CMOS) qui conserve son contenu grâce à une pile (sur des machines anciennes) ou une batterie (machines actuelles). Le programme permettant de configurer le BIOS est le ***SETUP***. En effet, le BIOS peut être prévu pour supporter plusieurs disques durs mais il faut les déclarer (type de disque, unité logique -C: ou D:.., nombre de cylindres...). Les informations qui seront fournies (nombre et type de disques, nombre de lecteurs de disquettes, extension mémoire,...) seront stockées dans le CMOS.

Intel prévoit un maximum de périphériques dans ses prochains BIOS ainsi que différents modes de communication avec ces matériels (CD ROM, écran graphique XGA, carte son...).

1.5 CLASSIFICATION DES SYSTEMES D'EXPLOITATION

Nous venons de voir l'architecture en couche d'un système d'exploitation ainsi que les différentes tâches que celui-ci doit assurer afin de permettre à l'utilisateur ou au programme de base d'exploiter les capacités de la machine d'une manière simplifiée.

Nous allons voir maintenant de quelle manière nous pouvons classer les systèmes d'exploitation en fonction de leurs possibilités, c'est à dire en fonction de la manière dont ils assurent les tâches décrites auparavant.

1.5.1 CLASSIFICATION PAR MODE DE TRAITEMENT (MONO, MULTI-UTILISATEUR)

1.5.1.1 TRAITEMENT PAR LOTS (BATCH PROCESSING)

Ce mode de traitement est le plus ancien et le plus classique. Il est encore largement utilisé. L'utilisateur donne un ensemble de programmes à exécuter (jobs) et ces jobs sont réalisés sans intervention possible de l'utilisateur. Ce dernier ne récupérant ses résultats que plus tard. Ce mode de traitement exige donc que toutes les données nécessaires au traitement soient définies par avance.

On utilise souvent un calculateur auxiliaire dont le rôle consiste à effectuer des conversions de supports (carte à bande et bande à imprimante) et gérer les files d'attente.

Avec les calculateurs de troisième génération dotés de la multiprogrammation (voir plus loin), les opérations de conversion de support se font en simultanéité avec le traitement des programmes. La différence importante réside dans le fait que le traitement par lot est remplacé par le chargement continu des travaux au fur et à mesure de leur arrivée. Les travaux sont mis en attente sur disque magnétique, puis chargés en mémoire pour être exécutés sous le contrôle du système d'exploitation qui tient compte de leur priorité.

1.5.1.2 TRAITEMENT INTERACTIF

L'utilisateur peut à tout moment intervenir dans la session courante ; il lance l'exécution de commandes, en apprécie les résultats, et peut décider lui-même de la suite des opérations.

1.5.1.3 TRAITEMENT EN TEMPS REEL (REAL TIME)

Ce mode de traitement est principalement utilisé pour des applications nécessitant un suivi de grandeurs physiques (guidage d'engins, suivi de chaîne, surveillance de centrales nucléaires, conduite

d'engins spatiaux, ...). Les données transmises à l'ordinateur sont traitées immédiatement. L'opération ainsi effectuée s'intitule ***contrôle de processus***.

Les ordinateurs temps réel ont des possibilités d'entrées-sorties spéciales ainsi qu'un système d'interruptions permettant au calculateur de se synchroniser immédiatement sur le processus à conduire ou à contrôler et, ainsi, d'être en mesure de réagir instantanément aux contraintes extérieures. On dit que les calculateurs travaillent ***en ligne*** pour exprimer le fait qu'ils sont connectés avec l'extérieur.

Le contrôle exercé sera en boucle ouverte lorsque l'homme peut, à partir des événements extérieurs, réagir sur le processus en cours (ex : destruction de fusée hors trajectoire) ; le contrôle est en boucle fermée si le calculateur réagit directement sur le processus au moyen d'organes de sortie spécialisés (ex : sirène d'un système d'alarme).

Le temps réel est un mode de traitement de données dans lequel les données proviennent, par messages séparés, d'un processus extérieur qui impose des contraintes de temps sur les délais d'entrée de chaque message et sur les délais de sortie des résultats correspondants.

Caractéristiques essentielles de ce mode de traitement :

- Les données arrivent dans un ordre arbitraire et à des instants aléatoires,
- L'information subit une dégradation quasi-instantanée si elle n'est pas immédiatement traitée,
- Les programmes peuvent changer en fonction du temps selon les valeurs des données extérieures.

1.5.1.4 TRAITEMENT EN TEMPS PARTAGE (TIME SHARING)

Un système est mono-utilisateur lorsqu'il ne peut gérer qu'un seul utilisateur. Il est Multi-Utilisateurs lorsqu'il est capable de distinguer les actions de plusieurs utilisateurs. On dit aussi qu'il est en ***temps partagé*** : Ceci correspond au cas ou plusieurs utilisateurs sont simultanément connectés à l'ordinateur. Le système d'exploitation permet alors de partager l'ensemble des ressources matérielles et logicielles en accordant à chacun un quantum de temps, de telle sorte que chaque utilisateur ait l'impression d'avoir son propre ordinateur à sa disposition. Il gère l'exécution des programmes.

Dans un système Multi-Utilisateurs apparaît la notion d'***administrateur***. Il est en effet primordial qu'un utilisateur particulier configure les environnements de chaque utilisateur et définisse les droits et propriétés de chacun.

Caractéristiques d'un système en temps partagé (time sharing system) :

- Nombre d'utilisateurs admis à utiliser le système en même temps (plus les utilisateurs sont nombreux, plus la machine passera de temps à permuter entre les utilisateurs),
- Valeur des temps de réponse admis pour chaque utilisateur,
- Limitations imposées à chaque utilisateur,
- Priorités entre les utilisateurs,
- Nombre total d'utilisateurs admis à partager le système.

1.5.2 CLASSEMENT PAR MODE DE PROGRAMMATION (MONO, MULTI-TACHE)

1.5.2.1 MONOPROGRAMMATION (MONOTACHE)

Chaque programme est exécuté séparément. L'utilisateur dispose de l'ordinateur tout entier à un instant donné.

Problème **:** lors d'opérations d'entrée-sortie, le processeur est obligé d'attendre un temps non négligeable, ces opérations étant très longues par rapport au temps de calcul (de l'ordre de 1 pour 1000).

Solution : donner la main à un autre programme pendant le temps d'attente pour entrée-sortie, afin de perdre le moins de temps possible. Cela s'appelle la multiprogrammation.

Multiprogrammation (multitâche)

Plusieurs programmes sont exécutés à la fois, ils sont présents en mémoire centrale, mais un seul d'entre eux est actif à un moment donné. Il pourra être interrompu à un autre moment par un autre programme, et sera alors mis en attente, pendant que l'autre sera exécuté par le processeur.

La commutation entre les programmes peut-être assurée soit par les entrées-sorties, soit de façon périodique par une horloge interne (meilleure méthode).

1.5.2.2 MULTITRAITEMENT

Plusieurs programmes sont exécutés par autant de processeurs distincts qui se partagent l'ensemble des ressources de l'ordinateur. Il ne s'agit donc plus de partage de temps sur un unique processeur, comme en multiprogrammation. Chaque unité centrale est allouée pour une exécution donnée.

Cas particulier de multitraitement : des processeurs spécialisés ou des coprocesseurs (DSP...) exécutent une partie d'un programme afin de décharger le processeur principal.

1.5.3 SYSTEMES D'EXPLOITATION OUVERTS OU FERMES

Un système d'exploitation fermé est un système dont les caractéristiques sont liées à la machine. On ne peut utiliser ces systèmes que sur certains ordinateurs particuliers.

MS-DOS est un système fermé : il ne peut exploiter que le processeur Intel (IBM P.C. et compatibles) ou d'autres processeurs à l'architecture quasiment identique.

UNIX est un système ouvert. Moyennant quelques modifications dans ses couches les plus basses, il peut se porter sur diverses machines.

CLASSIFICATION DE QUELQUES SYSTEMES D'EXPLOITATION

Caractéristiques	**MS-DOS**	**System 7**	**OS/2**	**Windows 95**	**Windows NT**	**Unix**
Fermé	X	X	X	X		
Ouvert			X*		X	X
Monotâche	X	X				
Multitâches			X	X	X	X
Mono-utilisateur	X	X	X	X	X	
Multi-utilisateurs						X
Temps réel						X
Multiprocesseurs					X	X

* en cours de réalisation au moment de la rédaction du document

Nous allons maintenant voir quelques-unes des tâches essentielles du système d'exploitation :

- La gestion des entrées-sorties.
- La gestion des interruptions.
- La gestion de la mémoire.
- La gestion des travaux et des ressources.
- La gestion de l'environnement et des erreurs.
- La gestion des fichiers.
- La gestion des programmes.

2. LA GESTION DES ENTREES-SORTIES ET DES INTERRUPTIONS

2.1 LES ENTREES-SORTIES

Nous avons vu que l'un des rôles fondamentaux de l'ordinateur est de récupérer des données, de les traiter puis de restituer les résultats souhaités à un utilisateur.

Si le traitement des données est une opération totalement interne à l'ordinateur, il n'en va pas de même pour les deux autres opérations. Il est cette fois indispensable de pouvoir communiquer avec le monde extérieur pour recevoir les données à traiter et pour restituer les données traitées.

Ces opérations prennent le nom ***d'entrées-sorties***.

2.1.1 PROBLEMATIQUE DES ENTREES-SORTIES

Les points suivants sont les problèmes majeurs à prendre en compte en ce qui concerne les échanges de données entre l'ordinateur et ses périphériques.

2.1.1.1 Une Très Grande Différence Entre La Vitesse De Traitement Et Les Différentes Vitesses De Transfert Propres Aux Périphériques

L'origine même des premiers systèmes d'exploitation réside dans la différence de vitesse entre l'unité centrale et les entrées-sorties.

Les entrées-sorties sont toujours très lentes comparées à la vitesse de calcul de l'unité centrale :

- Un processeur Pentium Risc cadencé à 100 Mhz traite globalement une instruction par cycle d'horloge. Il peut donc, dans l'absolu, transférer 10^8 instructions par seconde soit l'équivalent d'un livre moyen.
- Un modem rapide fonctionnant en norme V34 (28800 bps) peut transférer 3800 caractères par seconde.
- Une imprimante laser moyenne édite environ 5 pages par minute.

Pour éviter de faire attendre le processeur pendant qu'une opération d'entrée-sortie se déroule, ce qui revient à bloquer le processeur, le système d'exploitation se décharge de l'opération en la confiant au BIOS qui utilise un processeur spécialisé. Ainsi, le processeur central peut continuer à travailler pendant que l'opération d'E/S se déroule. Une fois cette dernière terminée, le BIOS avertira le système d'exploitation que l'opération est achevée. Le BIOS a donc pour rôle de conduire les opérations d'E/S standards.

Rappel : Ce n'est pas le système d'exploitation qui donne les ordres aux périphériques, mais le BIOS ou un complément du BIOS.

Entre le Système d'exploitation et le périphérique se logent deux interfaces dont le rôle est d'assurer l'indépendance entre le matériel et le logiciel : une ***interface logicielle*** et une ***interface matérielle***. Cette indépendance libère le programmeur des questions «bassement matérielles» : son application tournera aussi bien sur un disque dur 80Mo que sur un disque de 1Go, s'affichera toujours de façon identique quel que soit l'écran...

2.1.1.2 Les Problèmes De Synchronisme

Si l'on veut interrompre un ordinateur pendant un traitement quelconque, il faut le lui faire savoir. En règle générale, cela se fait par l'intermédiaire du clavier, encore faut-il qu'il existe un système de détection car, sans ce système, l'unité centrale n'a aucun moyen de savoir quand une touche est frappée. Il en va de même pour un jeu qui se déroule lorsque l'on actionne la souris ou la manette de jeu.

2.1.2 NOTION D'INTERFACE

Chaque périphérique, quel qu'il soit, est connecté à l'ordinateur par l'intermédiaire d'une ***carte interface***. Cette carte électronique est chargée de transformer les signaux électriques provenant du périphérique en signaux électriques compréhensibles par l'unité centrale et inversement.

Cette carte prend le nom de contrôleur (ou coupleur), elle est complétée par une partie logicielle que l'on nomme pilote (ou driver ou handler).



2.1.2.1 Interface Matérielle : Le Contrôleur De Périphérique

Le contrôleur de périphérique est un circuit de commande associé au matériel. Quelle que soit l'architecture du matériel périphérique, le contrôleur devra dialoguer avec l'unité centrale selon des normes. Le contrôleur est situé soit sur le périphérique, soit dans l'unité centrale sous forme d'une carte ou d'un composant sur la carte mère. La carte contrôleur peut aussi être rajoutée sur la carte mère : carte son, carte graphique...

Exemple : Le contrôleur d'écran.

Le contrôleur d'écran lit dans la mémoire les caractères à afficher et génère des signaux pour moduler le faisceau d'électrons qui dessine les caractères à l'écran. Sans ce contrôleur, le programmeur d'un système d'exploitation aurait à programmer directement le balayage du tube cathodique alors qu'il suffit au système d'exploitation d'initialiser le contrôleur avec quelques paramètres, comme le nombre de caractères par ligne et le nombre de lignes par écran pour qu'il se charge de diriger le faisceau.

Le contrôleur communique avec le système d'exploitation par l'intermédiaire de quelques registres. Le système d'exploitation effectue des E/S en écrivant des commandes dans les registres des contrôleurs.

Le contrôleur disque de l'IBM P.C., par exemple, accepte 15 commandes différentes comme READ, WRITE, FORMAT.... Dès qu'une commande est acceptée, ***le processeur peut effectuer un autre travail, le contrôleur s'acquittant seul de la commande***. Chaque unité périphérique possède donc un processeur spécialisé.

2.1.2.2 Interface Logicielle : Le Pilote De Périphérique

2.1.2.2.1 PRINCIPE

Chaque périphérique obéit aux ordres fournis par un contrôleur. Puisque chaque fabricant crée son propre contrôleur, il est bien évident qu'il n'est pas possible au système d'exploitation, ni même au BIOS, de fournir les commandes à tous les contrôleurs de périphériques.

Nous avons vu que chaque contrôleur possède des registres de commande. Le pilote (driver) est le logiciel qui connaît l'emplacement des registres du contrôleur et leur utilisation.

Le driver, comme le BIOS, permet le pilotage des périphériques : le *driver* est plus complexe mais *spécialisé* dans le pilotage d'un seul type de périphérique.

Le BIOS est un ensemble de fonctions élémentaires de pilotage standard. Le driver comporte des fonctions élaborées pouvant ou non utiliser certaines fonctions élémentaires BIOS. Les fonctions des drivers sont appelées ***routines***.

Chaque driver est fourni par le constructeur du périphérique sur une disquette. Pour pouvoir utiliser ce périphérique, il faut *au préalable installer le driver et le faire reconnaître* du système d'exploitation (sous MS-DOS, il suffit de rajouter une ligne dans le fichier CONFIG.SYS ou AUTOEXEC.BAT. Avec UNIX, la procédure est plus complexe car il faut incorporer le driver au noyau).

Sous DOS, le fichier IO.SYS contient des drivers standards qui accèdent aux fonctions BIOS pour piloter les périphériques standards, par exemple le pilotage d'une imprimante en mode texte.

Les drivers additionnels sont chargés en mémoire au démarrage de la machine et y demeurent pour être accessibles immédiatement en cas de besoin.

2.1.2.2.2 Fonctionnement

Lorsqu'un programme demande une entrée-sortie, une routine du sous-système d'entrées-sorties dirige la requête vers le driver après avoir interrogé certains registres de la machine afin de connaître l'adresse du driver demandé. Le driver traduit alors la requête d'entrée-sortie en termes concrets (actions physiques / actions électriques). Un driver disque doit, par exemple, déterminer où se trouve le bloc de données requis, vérifier si le moteur du disque tourne, si le bras est bien positionné, ... En bref, il doit déterminer les opérations que le contrôleur doit exécuter ainsi que l'ordre dans lequel il faut les exécuter.

Les programmes exécutent leurs entrées-sorties en indiquant au système un nom logique (A:, pour une disquette, C: pour un disque dur, ...), et le système maintient une table de correspondance entre les noms logiques et les drivers associés. Ceci assure une certaine indépendance entre le logiciel et le matériel puisque le dialogue est uniformisé du côté système d'exploitation. Le driver dialogue avec le matériel directement ou par l'intermédiaire du BIOS (pour communiquer avec des périphériques standard).

Le driver permet au système d'exploitation de piloter un périphérique en mode évolué (écran en mode XGA, imprimante en mode graphique) ou de piloter un périphérique additionnel (carte son, modem, ...) non prévu dans le BIOS.

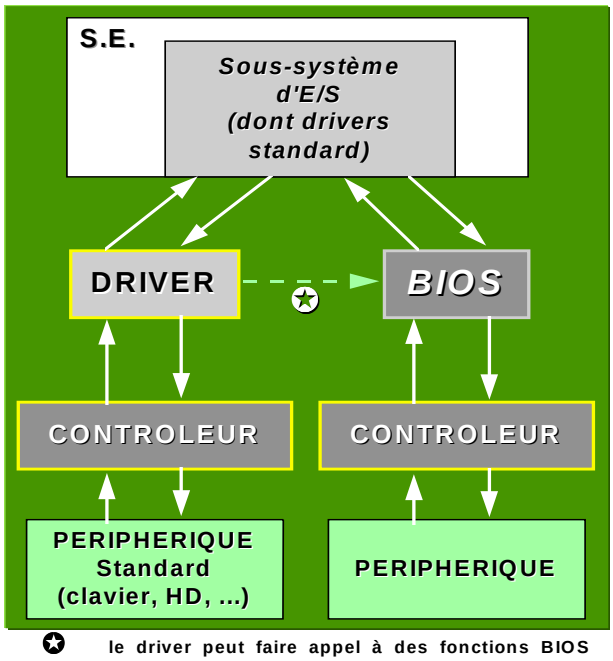
En somme, le système ***sous-traite le pilotage*** du périphérique au driver.

Nous considérerons que le BIOS et le driver constituent **l'interface d'Entrée/Sortie.**



2.1.2.3 Prise En Charge Par Le Système

Le système d'exploitation gère les E/S grâce à son sous-système de gestion des E/S qui doit lui permettre d'assumer les tâches suivantes : assurer une réponse rapide aux entrées-sorties, assurer l'indépendance logiciel-matériel via le BIOS ou directement par le driver spécifique.



Le Système d'exploitation et l'interface des E/S communiquent par ***interruptions***.

2.1.3 LES DIFFERENTS TYPES DE LIAISONS

Il existe essentiellement deux types de liaison entre l'ordinateur (le coupleur) et le périphérique. Elles dépendent du type de périphérique utilisé. Ces deux types de connexion sont la connexion série et la connexion parallèle.

2.1.3.1 La Connexion Série

La connexion série réalise l'interconnexion avec les unités périphériques de type série. Dans ces unités, les bits constituant la donnée sont émis les uns après les autres sur un seul fil de façon séquentielle dans le temps.

Un des problèmes les plus difficiles à régler, dans ce cas, réside dans la synchronisation entre émetteur et récepteur. Ont été ainsi définis des protocoles qui répondent chacun à des besoins ou des impératifs particuliers :

- transmission série asynchrone,
- transmission série synchrone,
- transmission série semi-synchrone.

2.1.3.2 La Connexion Parallèle

La connexion parallèle réalise l'interconnexion avec les unités périphériques qui émettent et reçoivent les données en parallèle. Dans ces unités, les bits constituant la donnée sont émis simultanément, un fil étant utilisé pour chaque bit. Le bus de connexion comporte donc autant de fils qu'il y a de bits dans la donnée.

Ces circuits parallèles sont utilisés pour des transferts de données qui doivent être rapides, par exemple, entre la mémoire de masse et les imprimantes rapides.

2.2 COMPLEMENT SUR LES ENTREES-SORTIES

2.2.1 SIMULTANEITE ENTRE TRAITEMENT ET ENTREES-SORTIES

Le problème le plus important à résoudre consiste à adapter des vitesses différentes entre les deux extrémités d'un processus d'entrée-sortie. En outre, le plus précieux lors d'un traitement quelconque est le temps processeur. On a donc cherché des méthodes permettant au processeur de faire autre chose pendant une entrée-sortie.

Fondamentalement, il existe deux techniques (avec variantes) différentes :

- Le mode par test d'état (***polling***) :

  On inclue (le système en est chargé) dans les programmes, avec une périodicité correcte, des points de test permettant de savoir si l'unité périphérique est prête à réaliser l'opération souhaitée. Cette technique est très lourde et sert surtout dans les cas de périphériques partagés, de réseaux, ...

- La méthode par interruption de programme :

  Au lieu de truffer les programmes de tests d'état des périphériques, on utilise la technique des interruptions (voir plus loin). L'unité centrale est ainsi déchargée de la surveillance périodique et coûteuse en temps des périphériques, ces derniers prévenant l'unité centrale quand ils sont disponibles. Toutefois, même si le gain est appréciable, pendant le transfert d'un bloc d'information, le processeur sera totalement occupé et donc indisponible pour les déroulements de programmes.

2.2.2 L'ACCES DIRECT A LA MEMOIRE (DMA)

Dans les modes précédents, dès lors que le processus d'entrée-sortie est lancé, le processeur est occupé à faire des accès mémoire en lecture ou en écriture selon qu'il s'agit d'une entrée ou d'une sortie. De plus, si l'on veut transférer un seul mot, il sera nécessaire d'effectuer une opération de transfert complète qui prendra une vingtaine de cycles d'horloge pour si peu de transfert.

L'idée de base consiste donc à éviter le passage des données à transférer par l'unité centrale afin de la libérer. On a donc créé des unités spécialisées, Unités d'Accès Automatique à la Mémoire (DMA : direct memory acces) susceptibles de prendre complètement en charge le transfert de tout bloc d'information tout en permettant au processeur de faire autre chose.

Dans ce cas, on va le voir, la simultanéité entre programme et entrée-sortie est presque totale d'où un gain de temps très appréciable.

Le déroulement d'une opération d'entrée-sortie s'effectue de la façon suivante :

1 **- *Initialisation*** de l'opération par le programme. Il s'agit de fournie au processeur d'entrée-sortie les données concernant l'organisation de l'opération : nombre de données à transférer, adresse de la première, sens du transfert, vitesse de transfert, taille du tampon (buffer) intermédiaire, adresse du périphérique ...

2 **- *Transfert*** proprement dit au rythme du périphérique sous le contrôle de l'unité d'échange en utilisant différentes techniques selon le choix, l'urgence ou tout autre impératif:

- Transfert ***par bloc*** : les transferts se font par blocs avant de rendre la main à l'unité centrale qui peut alors continuer le transfert pour un autre bloc ou donner la main à un autre programme plus urgent et continuer le transfert ultérieurement.
- Transfert ***par vol de cycle*** : après les paramètres d'initialisation, les transferts se font par caractère quand le processeur est libre (on se souvient que, lors du déroulement d'une instruction, le processeur et la mémoire centrale ne sont pas actifs en même temps), on dit que l'on vole un cycle processeur. Lorsque les transferts ne sont pas urgents, il y a adaptation au système.

3 - Interruption de fin d'opération vers le processeur.

Ce système a ensuite été amélioré par l'***enchaînement automatique des travaux***. Précédemment, l'unité centrale devait initialiser chaque transfert en chargeant les informations nécessaires dans les registres de l'unité d'échange. Afin de soulager l'unité centrale, on a conçu l'unité d'échange comme une unité ayant ses propres capacités d'initialisation. Ainsi, pour tout transfert, l'unité centrale créera en mémoire un ***programme canal*** qui définit de manière exhaustive tous les transferts successifs qui seront à réaliser. Ce programme canal permet d'enchaîner les transferts sans intervention de l'unité centrale.

2.2.3 LES CANAUX D'ENTREES-SORTIES

Dans les gros ordinateurs (et les petits suivent désormais le même principe), la taille du système et le prix de l'unité centrale justifient le fait que tous les efforts portent sur l'optimisation du temps d'utilisation du processeur. La perte de temps liée aux entrées-sorties contrôlées par programme rend cette approche très difficile. En outre, l'efficacité du système DMA pousse les utilisateurs à faire toutes les entrées-sorties avec ce mode pour libérer un maximum de temps CPU. Mais un contrôleur DMA reste coûteux et complexe.

Le contrôleur DMA va donc être partagé entre plusieurs interfaces. C'est le concept de ***canal d'entrées-sorties***. Ce canal comporte un processeur spécialisé qui fonctionne comme un DMA mais qui présente la particularité d'être partagé entre plusieurs interfaces.

Toutefois, comme les canaux travaillent en parallèle avec le processeur central, il y à concurrence pour accéder à la mémoire centrale, les canaux devant lire et écrire dans la même mémoire que celle utilisée par l'unité centrale pour l'exécution du programme en cours. Les canaux doivent donc, comme vu précédemment, pouvoir voler des cycles d'accès mémoire au processeur central.

Une des améliorations sensibles apportées à ces canaux consiste à installer une mémoire tampon sur la carte d'interface. La carte peut être connectée à des périphériques très lents, elle sera donc utilisée comme tampon de stockage.

Un exemple : la carte contrôleur SCSI (Small Computer System Interface)

Un contrôleur SCSI permet d'améliorer sensiblement les performances d'un ordinateur mais il donne surtout accès à de nombreuses possibilités d'extension. En effet, une machine équipée d'un contrôleur SCSI peut recevoir jusqu'à sept unités de périphériques sans adjonction de carte contrôleur supplémentaire. Le contrôleur SCSI offre, en outre, de meilleures performances en matière de taux de transfert et de rapidité d'accès aux périphériques connectés. Un contrôleur de ce type est une carte d'extension prenant place dans les connecteurs standard de la machine.

On trouve des cartes de ce type pour chacun des standards de bus interne : ISA, EISA, VLB, PCI, ... Le contrôleur SCSI fait office de passerelle entre les bus internes de la machine et le bus SCSI de la carte.

2.3 LES INTERRUPTIONS

2.3.1 POURQUOI DES INTERRUPTIONS ?

Lorsqu'une machine est en marche, l'unité centrale va, par définition, toujours exécuter les instructions du programme en cours. Il faut cependant que le système d'exploitation prenne en compte

les événements extérieurs à la séquence d'instructions qu'il déroule (appui sur une touche du clavier, une imprimante vient de finir sa page, problèmes matériels...).

Il faut donc utiliser un mécanisme qui permette à l'interface de prévenir le système de l'arrivée de tels événements.

Le système d'exploitation et l'interface des entrées-sorties communiquent par le mécanisme des *interruptions (rupture de séquence)*.
Les interruptions permettent de prendre en compte des événements relatifs à des E/S.

A chacun de ces événements correspond un certain travail à exécuter, sans délais ou dans un délai fixé, par l'unité centrale. Ce travail est codé sous la forme de routines systèmes ou de programmes. Mais, pour exécuter ce nouveau travail, il faut qu'il y ait interruption du travail en cours, il faut qu'il y ait une rupture dans la séquence des instructions en cours d'exécution. De plus, une fois ce nouveau travail exécuté, l'unité centrale doit reprendre l'exécution de la séquence interrompue à l'endroit exact où elle l'a abandonné.

2.3.2 MECANISME D'INTERRUPTION

Le processeur possède une (des) broche(s) physique(s) grâce à laquelle (auxquelles) les événements extérieurs peuvent être pris en compte. Une tension significative sur la broche d'interruption est une ***demande d'interruption***. Le processeur va réagir à cette demande en activant le programme (routine) correspondant. L'interruption est appelée IRQ (Motorola) ou INT (Intel). C'est le contrôleur du périphérique qui envoie le signal sur la broche du processeur sur le bus qui relie ces deux éléments.

Lorsqu'une interruption est prise en compte par le processeur, les interruptions de même priorité ou de priorité inférieure à venir sont ignorées jusqu'à la fin du traitement de l'interruption en cours. Une routine d'interruption ne pourra être interrompue que par une interruption de ***priorité*** supérieure. Certains sous-programmes ne doivent pas être interrompus (par exemple l'écriture d'un secteur sur disque) sous peine de graves dysfonctionnements, ils porteront donc une priorité très forte.

Afin de prendre en compte une interruption, il faut évidemment que le processeur réagisse au changement d'état de la broche d'interruption.

2.3.3 DEROULEMENT D'UNE INTERRUPTION

Lorsqu'une interruption est émise, le processeur termine l'exécution de la dernière instruction en cours. Il sauvegarde ensuite son ***contexte*** (contenu de ses registres), puis il exécute la routine correspondant à l'interruption reçue. Pour trouver la bonne routine (en ROM ou en RAM), il fait appel à la ***table des interruptions*** (en RAM) qui contient l'adresse de la routine pour chacune des interruptions. Une fois l'exécution de cette routine terminée, il restaure son contexte et reprend son fonctionnement normal en reprenant le processus initial là où il avait été interrompu.

Définition : on appelle ***contexte*** d'un programme ou ***vecteur d'état*** l'ensemble des données système indispensables à la reprise d'un programme interrompu :

- Le contenu du compteur ordinal permettant de reprendre le séquencement des instructions du programme,
- Le contenu de la pile où se trouvent toutes les valeurs des différentes variables du programme,
- Le contenu des divers registres particuliers : accumulateurs, index, ...

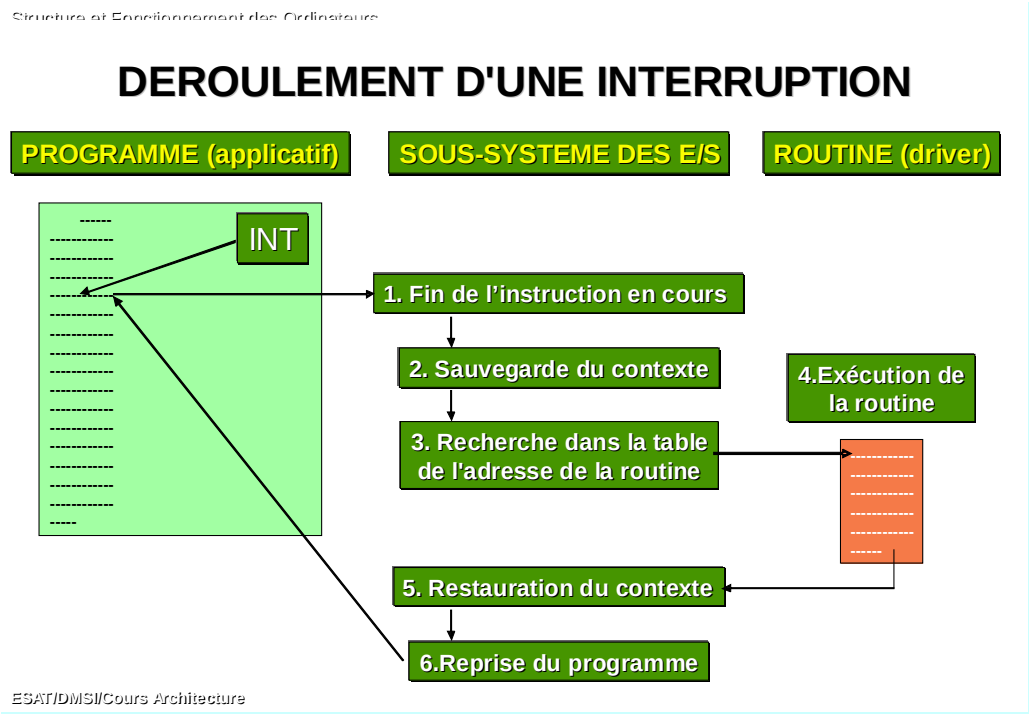
La sauvegarde du contexte est faite en général par logiciel, toutes les données sont rangées dans des zones spéciales.

Les routines d'interruption varient selon le type d'interruption. Des mécanismes permettent au processeur de se brancher sur la bonne routine. Le sous-système de gestion des entrées-sorties met à jour la table des interruptions en fonction de la configuration de la machine.

Un exemple : frappe de la touche 'A' sur le clavier
Cheminement : touche ➔ connecteur clavier ➔ broche du microprocesseur.

1 - Le processeur prend connaissance du type d'interruption : elle peut être prise en compte (si elle ne l'est pas, le processeur reprend son travail : tout dépend de la priorité de l'interruption par rapport au travail en cours).
2 - Le processeur sauvegarde le contexte
3 - Lecture de la table des interruptions : chargement de l'adresse de la routine de gestion du clavier.
4 - *Début de la routine : récupérer le code géographique de la touche dans le clavier, calculer le caractère correspondant, mettre le caractère dans le buffer, avertir le processeur que l'opération est terminée, fin de la routine.*
5 - Restauration du contexte.
6 - Reprise du programme principal. Ici, la routine est une suite de fonctions élémentaires (en italiques ci-dessus) du BIOS (pilotage du clavier).

D'une manière générale, le mécanisme d'interruption se présente comme suit :



La routine donne les ordres au périphérique et redonne la main au programme principal. On n'attend pas que le périphérique ait fini son travail pour reprendre le travail du processeur central : le processeur spécialisé du périphérique travaille en même temps que l'unité centrale. Lorsqu'il a terminé, le périphérique provoque une nouvelle interruption pour que le processeur central en tienne compte.

2.3.4 CLASSIFICATION DES INTERRUPTIONS

On classe généralement les interruptions en deux grandes catégories : les interruptions externes et les interruptions internes.

2.3.4.1 LES INTERRUPTIONS EXTERNES

Les interruptions externes sont dues aux périphériques ou à des événements extérieurs au système informatique. Pour signaler à l'unité centrale qu'ils sont prêts, les périphériques émettent un signal matériel (changement d'état logique) qui déclenche directement l'exécution de la routine d'interruption.

Exemple : mise en service d'un périphérique, fin de lecture ou d'écriture d'une piste, touche appuyée au clavier, incident matériel, etc.

Ce sont des signaux matériels qui parviennent sur les broches du processeur. Ce dernier positionne alors à 1 un ou plusieurs bits internes dont l'un est scruté à la fin de l'exécution de chaque instruction. L'unité centrale termine donc l'instruction en cours avant de traiter l'interruption.

2.3.4.2 LES INTERRUPTIONS INTERNES

Les interruptions internes peuvent être de plusieurs sortes : interruptions d'horloge, interruptions dues à des erreurs système, ...

- Les interruptions d'horloge (par convention) permettent la gestion du temps par le système d'exploitation : toutes les N milli-secondes, l'interruption d'horloge fait appel à l'ordonnanceur qui commute les tâches ; elle sert aussi à régler certains événements propres au système.

- Les interruptions dues à des erreurs système : dépassement de capacité d'un registre ou d'une zone, violation d'accès mémoire, exécution d'une instruction illégale, ...

Si ces interruptions interviennent pour empêcher une instruction erronée de s'exécuter complètement, alors elles sont traitées pendant l'instruction en cours. Elles déclenchent ce que l'on appelle une séquence de ***TRAP*** (séquence d'interruption qui ne met pas à jour le compteur ordinal).

2.3.5 LES INTERRUPTIONS LOGICIELLES

Cette appellation est abusive. Elle signifie simplement qu'il existe une instruction ayant les mêmes effets qu'une interruption matérielle. La seule différence est qu'il n'y a pas de signal d'interruption sur aucune broche (système réservé aux périphériques).

Grâce aux interruptions logicielles, le programme peut faire appel au système d'exploitation pour réaliser des entrées-sorties (écriture sur disque, prise en compte d'une donnée entrée au clavier par l'utilisateur...). Le système d'exploitation fera alors appel aux routines qui correspondent.

Pour assurer la sûreté de fonctionnement d'un système d'exploitation, l'unité centrale doit pouvoir gérer deux modes : le ***mode système*** (ou mode superviseur ou mode maître) et le ***mode utilisateur*** (ou mode esclave).

Comme le système d'exploitation prend en charge les entrées-sorties, les programmes utilisateurs ne doivent pas pouvoir effectuer directement de transfert physique. Il existe donc des instructions, appelées ***instructions privilégiées***, qu'il est impossible d'exécuter en mode utilisateur

(celui dans lequel se déroulent les programmes utilisateurs), de même qu'il peut exister des zones mémoires interdites (registres périphériques par exemple). Par contre, le système d'exploitation, auquel tout est permis, fonctionne entièrement en mode système. C'est donc par ce moyen que s'effectue le dialogue entre le programme utilisateur et le système d'exploitation.

Une interruption logicielle est, en fait, ***un appel au système*** par l'intermédiaire d'une instruction machine spéciale (TRAP sur famille 68000, INT sur 8086) qui simule directement une interruption (en général moins prioritaire que les interruptions matérielles), et qui passe donc directement en mode système. Ceci s'effectue après passage de paramètres par le programme souhaitant faire appel au système.

On passe donc du mode utilisateur au mode système par l'intermédiaire des interruptions en réalisant des appels système.

2.3.6 EXEMPLES

2.3.6.1 PRINCIPE GENERAL

Le programme demande l'exécution d'une E/S par une interruption logicielle. Un appel à une primitive système d'E/S est alors déclenché. Cette primitive vérifie les paramètres de l'E/S et transmet l'appel au driver correspondant. Le driver envoie le premier message pour démarrer l'E/S ; le périphérique signale par une interruption matérielle qu'il est prêt à recevoir le message suivant, la routine driver signale que l'opération d'E/S est terminée et le contrôle est rendu à la primitive (première restauration du contexte) puis au programme appelant (deuxième restauration du contexte). ***Une interruption logicielle appelle à son tour une interruption matérielle.***

2.3.6.2 LE CLAVIER SOUS MS/DOS

L'appui sur une touche de clavier déclenche une interruption matérielle qui appelle une routine (driver clavier). Comme vu précédemment, cette routine calcule le code ASCII et stocke les informations dans le buffer clavier.

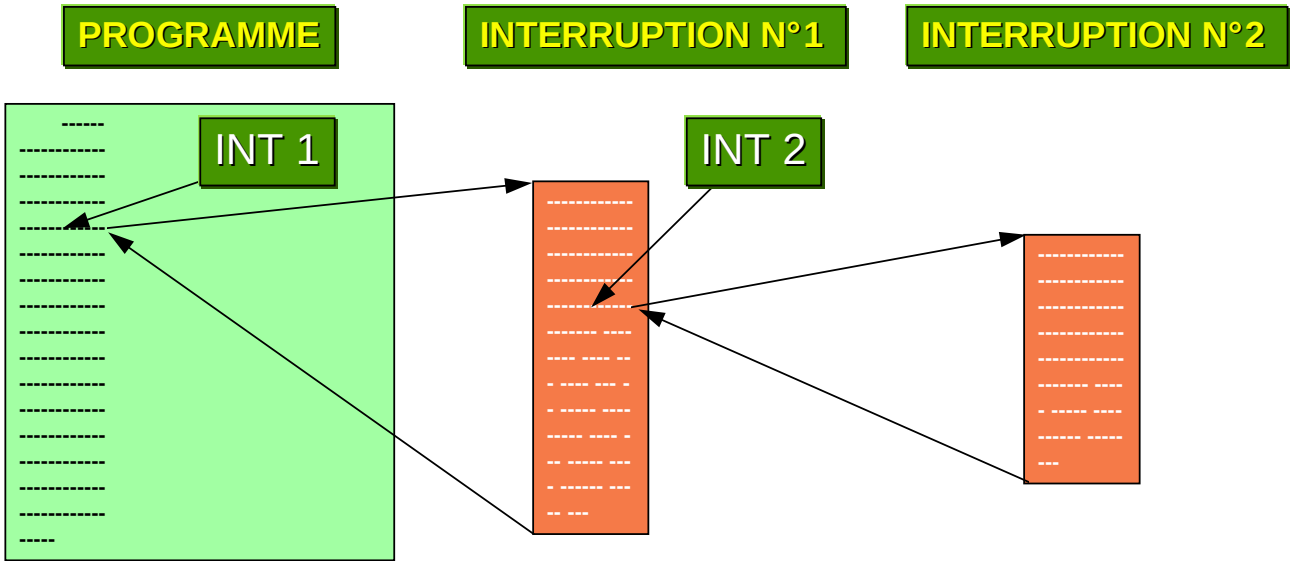
Le système ou l'utilisateur peut alors aller lire le buffer clavier par l'intermédiaire d'une interruption logicielle.

Il faut aussi remarquer que le relâché de la touche, déclenche, au même titre que l'appui, une procédure similaire avec un code géographique différent. Ceci est indispensable pour que le système puisse prendre en compte le relâché de certaines touches comme SHIFT par exemple, qui peuvent rester appuyées pendant que l'utilisateur frappe d'autres touches.

2.3.7 LA PRIORITE DES INTERRUPTIONS

Les interruptions n'ont pas toutes la même importance. Pour traduire ce fait, on leur attribue un niveau de priorité, elles sont hiérarchisées. Un programme ou une routine d'interruption ne pourra être interrompu que par une interruption de niveau supérieur. A chaque niveau d'interruption sera associée une zone de sauvegarde pour les contextes correspondants.

Cas des cascades d'interruptions :



L'interruption n°1 ne peut interrompre l'interruption n°2 que si sont niveau de priorité est supérieur. On comprend bien ici qu'il faudra une (et une seule) zone de sauvegarde par niveau d'interruption.

2.3.8 EXEMPLE : LES INTERRUPTIONS SOUS MS-DOS (CAS DU P.C. AT)

Comme précédemment, on distingue des interruptions logicielles et des interruptions matérielles.

2.3.8.1 LES INTERRUPTIONS LOGICIELLES

Les interruptions logicielles sont déclenchées par un programme se déroulant pour exécuter une fonction particulière du DOS ou du BIOS. Pour déclencher cette fonction, il n'est pas nécessaire de connaître l'adresse de la routine correspondante, il suffit de connaître le n° de l'interruption souhaitée. Ces numéros sont standardisés.

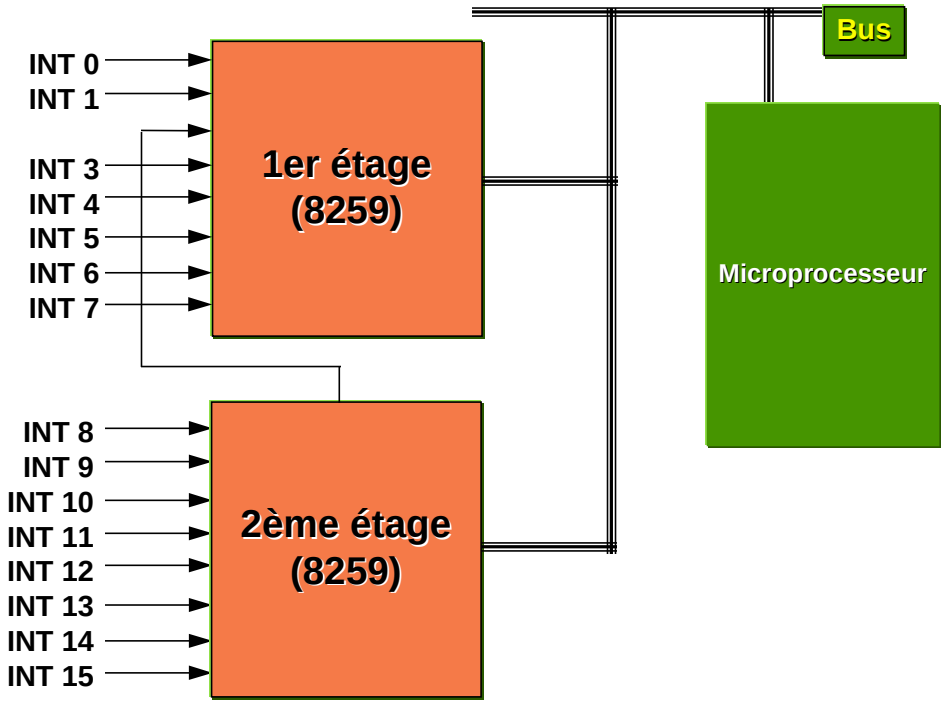
2.3.8.2 LES INTERRUPTIONS MATERIELLES

A l'inverse des précédentes, ce n'est pas le programme qui les déclenche mais les périphériques : clavier, imprimante, lecteur de disquette, lecteur de disque, ...

Tous les périphériques sont connectés à un contrôleur d'interruptions qui comporte entre autre un processeur spécialisé de type 8259.

sera égal à la taille d'une page. Dans la pagination, la mémoire

Le P.C. de type AT possèdent 2 processeurs 8259 mis en cascade ce qui leur permet de gérer 15 interruptions (le P.C. de type XT possédaient un seul 8259 et géraient 8 interruptions). Ceci est illustré par le schéma ci-dessous :



Dans ce cas, les correspondances d'interruption sont les suivantes (de la plus prioritaire à la moins prioritaire) :

N° d'interruption	N° d'INT dans le 8259	Fonction
0	0 / 1	Timer
1	1 / 1	Clavier
2	2 / 1	Cascade 2ème 8259
8	0 / 2	Horloge temps réel
9	1 / 2	
10	2 / 2	
11	3 / 2	
12	4 / 2	
13	5 / 2	coprocesseur mathématique
14	6 / 2	disque dur
15	7 / 2	
3	3 / 1	2ème port série
4	4 / 1	1er port série
5	5 / 1	2ème port parallèle
6	6 / 1	lecteur de disquette
7	7 / 1	1er port parallèle

Les entrées non utilisées sont disponibles pour les cartes d'extension (SCSI, lecteur CD, streamer, ...).

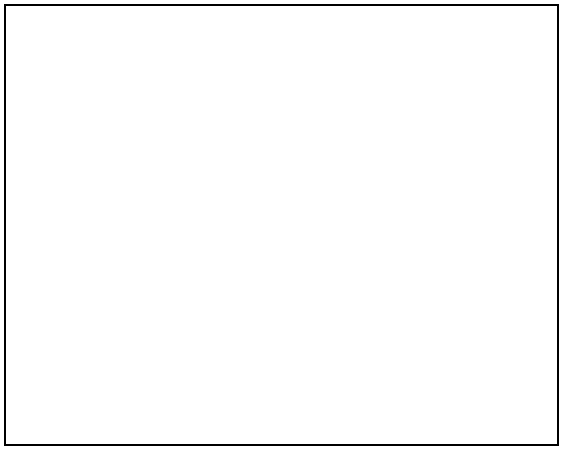
2.3.8.3 STRUCTURE DE LA TABLE DES VECTEURS

On parle de table des vecteurs d'interruption car il en existe 256. Les interruptions logicielles et matérielles sont mises sur le même plan, seule l'origine de l'appel les distingue.

Chaque élément (entrée) de la table occupe deux mots successifs, la table s'étend donc sur 1024 octets (un mot = 2 octets ; une entrée = 4 octets ; table = 256 x 4 = 1024 octets). Pour chaque interruption, le premier mot représente l'adresse du début de la routine, le deuxième mot représente son ***offset*** (longueur).

Il est important de noter que cette table est chargée en totalité en mémoire centrale au démarrage du système et qu'elle peut donc être modifiée par n'importe quel programme. Ceci peut s'avérer très dangereux pour l'utilisateur non averti mais très intéressant pour les autres. A titre indicatif, de nombreux virus fonctionnent par détournement d'interruption.

Exemple de distribution des interruptions sur un i486 :



3. LA GESTION DE LA MEMOIRE

3.1 POURQUOI UNE GESTION DE LA MEMOIRE ?

La gestion de la mémoire centrale (RAM) par le système d'exploitation vise cinq buts.

3.1.1 SIMULATION D'UNE MEMOIRE INFINIE

La mémoire centrale a toujours une capacité restreinte, alors que les programmes deviennent de plus en plus demandeurs. Le système d'exploitation doit assurer la simulation de cette mémoire de façon à permettre l'***exécution de programmes dont la taille dépasse de loin la capacité de la mémoire centrale***.

Dans un système en multiprogrammation, un certain nombre de processus doivent se partager une seule mémoire centrale. Le système d'exploitation doit assurer la simulation de cette mémoire de façon à permettre l'***exécution simultanée processus de plus en plus nombreux.***

3.1.2 TRANSPARENCE VIS A VIS DES PROCESSUS

La gestion de la mémoire doit être transparente vis-à-vis du processus. Un processus peut à priori être chargé n'importe où en mémoire (en fonction de l'emplacement et de la taille des processus qui ont été chargés avant lui). Il est donc important que le code du processus (après édition des liens) ne soit pas modifié par le programmeur en fonction de l'endroit où le programme est placé, mais que, si modification il y doit y avoir, ce soit le système d'exploitation qui la réalise. Le mieux étant qu'aucune modification ne soit à réaliser.

3.1.3 PROTECTION DES PROCESSUS

Dans un ordinateur en fonctionnement coexistent au moins deux processus : l'un pour le système, l'autre (les autres) pour le(s) programme(s) en cours d'exécution. Pour éviter qu'un mauvais fonctionnement de l'un des processus ne provoque la destruction d'un autre processus, il faut l'empêcher d'aller écrire hors des zones de mémoire réservées.

3.1.4 GESTION DES PARTIES DE PROGRAMME

Un programme se compose de quatre parties appelées segments : instructions (code), données (data), pile (stack) et données du système (system data). Il s'agit de gérer les emplacements des segments de telle façon qu'ils forment des ensembles logiques quelles que soient leurs adresses réelles.

3.1.5 PARTAGE DE CODE

Un processus peut-être partagé par plusieurs processus pères en ne gardant qu'une copie du code en mémoire. Le système se charge alors de l'adressage multiple. (exemple : les Dynamic Link Library, DLL).

3.2 DEFINITIONS

3.2.1 MEMOIRE PHYSIQUE

C'est la mémoire centrale physiquement (puces, barrettes, cartes) présente dans l'ordinateur dont la taille effective varie maintenant de quelques Ko à plusieurs Go (gros systèmes). C'est dans cet espace mémoire et ***uniquement*** dans celui-ci que s'exécutent les programmes.

3.2.2 MEMOIRE VIRTUELLE

La mémoire virtuelle est constituée par l'espace mémoire adressable par un programme. Elle peut aller jusqu'à plusieurs dizaines de To (Tera octet).

Le nombre d'adresses virtuelles correspond au nombre fictif de cellules mémoires dont il faudrait disposer pour charger la totalité du programme et de ses données en mémoire centrale.

Si la mémoire physique est plus grande que la mémoire virtuelle (cas extrêmement rare), il suffit d'affecter à chaque programme une adresse de base en mémoire physique (adresse de la première instruction du programme), les adresses suivantes étant translatées (recalculées) par rapport à l'adresse de base en mémoire physique. Cette translation d'adresses porte le nom de mapping.

Dans la quasi-totalité des cas, l'espace mémoire virtuel est plus grand que l'espace mémoire physique. Il s'agit donc d'effectuer la correspondance entre adresses logiques de la mémoire virtuelle et adresses mémoires physiques. Cette correspondance est en général effectuée de façon matérielle par un circuit appelé MMU (Memory Management Unit).

Exemple : un ordinateur peut générer des adresses opérandes sur 16 bits (de 0 à 64Ko), ce sont des adresses virtuelles. Cet ordinateur n'a cependant que 32Ko de mémoire physique. On peut donc écrire des programmes de 64Ko mais on ne peut pas les charger entièrement en mémoire. Une image du programme doit alors être stockée sur disque de telle sorte qu'on puisse en charger, au besoin, différentes parties dans la mémoire centrale.

3.2.3 LE SWAPPING

Pour compenser le manque de mémoire physique, on va recourir à la mémoire de masse. En effet, à un instant donné, seule la partie du programme correspondant au processus actif a besoin d'être présente en mémoire centrale. Le système peut donc aller "décharger" sur la mémoire de masse les processus inactifs. Cette opération a pour nom ***swapping***.

Il existe deux méthodes principales de gestion de la mémoire : la segmentation et la pagination. Ces deux méthodes reposent sur le swapping.

3.3 LA SEGMENTATION

3.3.1 PRINCIPES

Dans la méthode de segmentation, le programme est découpé en segments qui correspondent chacun à une procédure (sous-programmes), un module logique ou un ensemble de données. Les segments sont de taille variable, définis et nommés par l'utilisateur (le programmeur). Un segment est

donc un sous-programme logiquement indépendant qui peut communiquer avec d'autres segments par des procédures d'appel avec passage d'arguments.

Chaque processus demande la présence d'un certain nombre de segments dans la mémoire centrale. A un moment donné, seuls les segments les plus "utiles" seront présents en mémoire centrale. Ceci est rendu nécessaire par le fait que la capacité de la mémoire centrale est limitée.

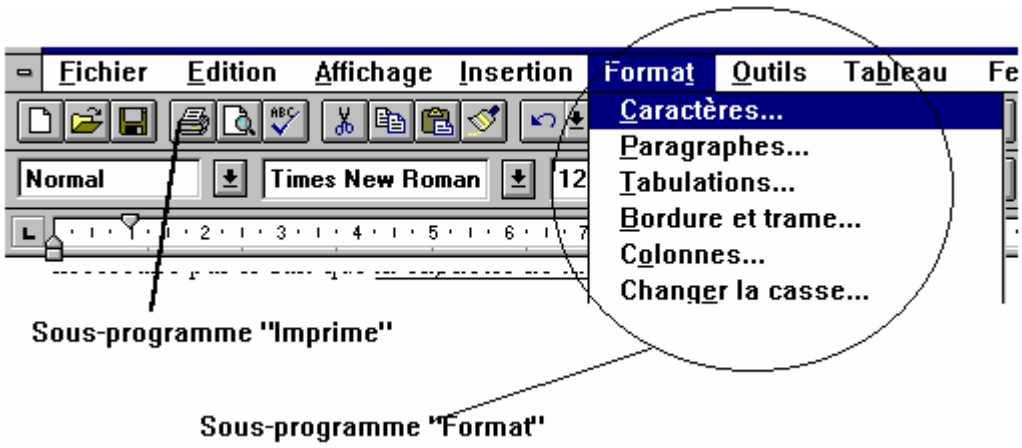
La mémoire centrale est elle-même divisée en segments correspondant à des unités d'adressage physique indépendantes les unes des autres. Chaque segment physique est une série d'adresses (de cellules mémoires) continues de 0000 à une adresse maximale. La taille d'un segment de programme sera donc comprise entre 0 et l'adresse maximale autorisée (le programmeur devra connaître sa machine). Par exemple, le 80386 d'Intel **peut gérer** au maximum 16000 segments d'un milliard de mots mémoires de 32 bits.

En général, la taille des segments est supérieure à leur nombre ; en effet, peu de programmes nécessitent plus de 1000 segments alors que certains programmes requièrent des segments de plusieurs méga-octets. Les premières machines ne possédaient qu'un seul segment en mémoire et tout nouveau segment chargé venait écraser l'ancien : le programmeur devait gérer les ***recouvrements*** (***overlays***).

Fort heureusement, bien que les machines modernes puissent gérer des milliers de segments, le programmeur ne sera pas obligé d'utiliser toute cette possibilité. S'il le souhaite, sa mémoire ne se découpera qu'en 10 segments. S'il a besoin de 100 segments, ils seront globalement 10 fois plus petits. Son choix devra seulement respecter les limites imposées par le constructeur, comme celles du i80386, par exemple.

Exemple : le traitement de texte Word.

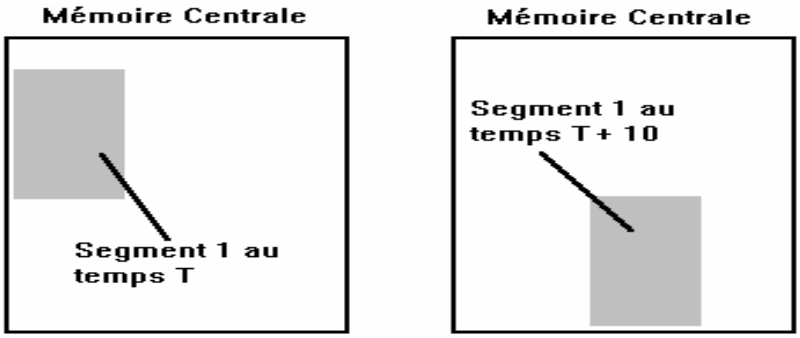
Lorsque l'on utilise Word, le programme Word n'est pas intégralement contenu en mémoire centrale, sauf si votre machine est suffisamment performante pour le charger d'un seul tenant. Seules les fonctions les plus utilisées sont adressées en RAM, les autres sont en attente sur disque.



Le sous-programme "Imprime" peut représenter un segment, comme "Format", ou encore "Fichier", etc... Sur certaines machines, il faut attendre quelques instants pour accéder à "Format/Caractères..." ou à d'autres fonctions : les segments de programme doivent d'abord "monter" en mémoire. Si quelques instants après on redemande "Format/Caractères", ce sera presque instantané : le segment est encore présent en mémoire centrale.

3.3.2 CHARGEMENT DES SEGMENTS EN MEMOIRE CENTRALE ET ADRESSAGE

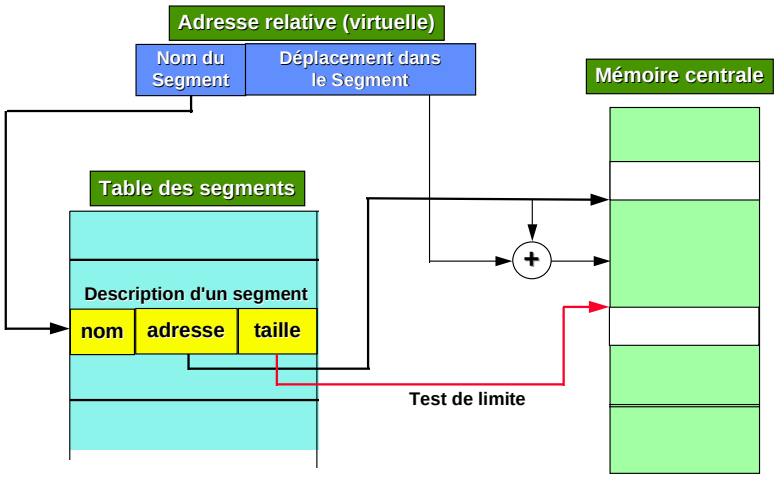
Selon le moment où il est chargé, le même segment pourra occuper des places différentes en mémoire centrale. Les adresses des instructions du segment seront donc variables. L'adresse physique de la première instruction du segment correspond à l'adresse du segment.



L'adressage des instructions du programme sera ***relatif au début du segment*** et non au début du programme. Chaque adresse relative (virtuelle) est composée d'un couple : **nom du segment + déplacement dans le segment** (offset).

Chaque programme segmenté possède une ***table des segments*** contenant une description de chaque segment. Chaque descripteur de segment contient : un champ nom, un bit de présence (ou d'absence), des bits de contrôle (lecture, écriture, ...), un champ adresse (en mémoire réelle ou virtuelle), un champ taille. Si le segment est en mémoire centrale, la table indiquera son adresse sinon le système ira le chercher en mémoire auxiliaire.

Si un programme tente d'accéder à un segment non présent en mémoire centrale et que la mémoire centrale est pleine, le système sauvegarde alors un segment "bien choisi" sur le disque pour le remplacer par le segment nécessaire.



3.3.3 AVANTAGES ET INCONVENIENTS DE LA SEGMENTATION

Pratique pour le programmeur, cette méthode permet une programmation structurée et modulaire. Chaque programme est découpé en unités (procédures) s'appelant les unes les autres. Chacune des unités peut être modifiée et recompilée indépendamment du reste du programme n'y

faisant pas référence. Un segment peut contenir soit du code (un sous-programme ou procédure), soit un tableau ou une suite de données mais pas un mélange d'objets de différentes natures. Le segment permet ainsi de séparer les instructions des données dans un souci de sécurité.

Le segment permet la multiprogrammation (le multitâche) : le fait de segmenter la mémoire permet d'exécuter plusieurs programmes à la fois ; chacun des programmes étant chargé dans son segment, indépendamment des autres.

Cependant, la taille variable des segments peut conduire à deux phénomènes bloquants :

a - l'effritement de la mémoire : le phénomène du damier.

Dans le schéma qui suit, supposons que le segment 1 (fig. (a)) soit retiré et remplacé par le segment 7 qui est plus petit.

Nous arrivons à la configuration de la mémoire de la figure (b) où Il existe un espace libre entre le segment 7 et le segment 2.

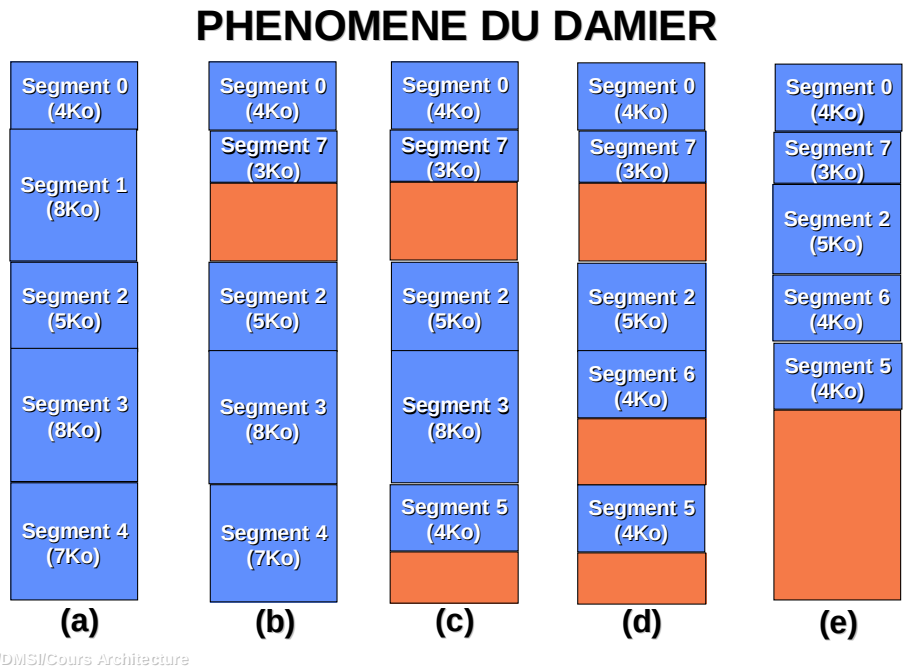
Le segment 4 est ensuite remplacé par le segment 5 à la fig. (c) et le segment 3 par le segment 6 comme le montre la fig. (d).

Après quelques temps, la mémoire est fragmentée en parties qui contiennent soit des segments soit des zones libres.

Ce phénomène appelé damier, gaspille de la mémoire dans les zones libres. En effet, ***un segment est contigu en mémoire***, il ne pourra donc pas se loger à différents endroits.

Le ***compactage*** permet de remédier à ce phénomène comme le montre la fig. (e). Mais le compactage prend du temps.

Structure et Fonctionnement des Ordinateurs



ESAT/DMSI/Cours Architecture

b - l'impossibilité, à un moment donné, ***de charger en mémoire un segment de grande taille***, sans un remaniement de la mémoire coûteux en temps : un segment serait d'une taille telle qu'il serait impossible de le charger malgré le compactage et le renvoi en mémoire auxiliaire du maximum de segments.

De plus, cette méthode d'optimisation de la mémoire demande au programmeur d'organiser son code en segments.

Il existe un remède aux inconvénients de la segmentation : la méthode de ***pagination***.

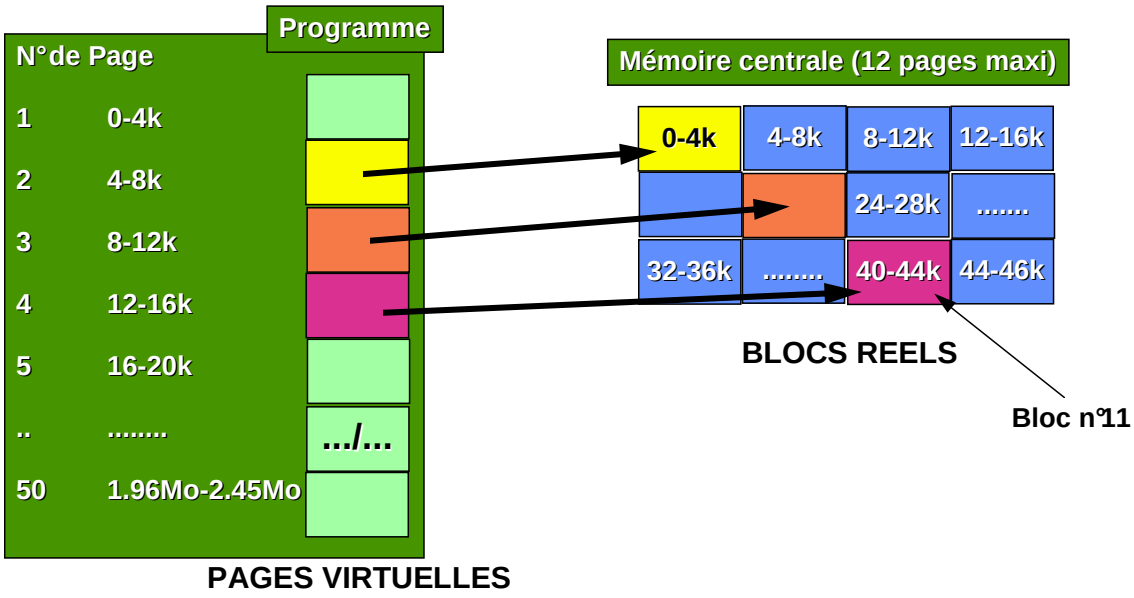
3.4 LA PAGINATION

3.4.1 PRINCIPE ET MISE EN OEUVRE

Une manière de réduire les inconvénients de la segmentation consiste à découper chaque programme en page de taille fixe.

Ainsi, chaque espace libre en mémoire pourra être utilisé pour charger une page puisque la taille d'un espace sera égal à la taille d'une page. Dans la pagination, la mémoire virtuelle (le programme) est découpée en pages de taille fixe (généralement de 512 octets à 8Ko).

L'espace physique est lui aussi découpé en blocs de même taille. Pour reprendre un exemple précédent, l'ordinateur de 64Ko de mémoire virtuelle et 32Ko de mémoire physique, on a 16 pages virtuelles de 4Ko et 8 blocs réels en mémoire centrale.



Comme pour la segmentation, chaque processus occupe un certain nombre de pages dans la mémoire virtuelle. A un moment donné, seules les pages les plus "utiles" sont présentes en mémoire centrale.

Les adresses manipulées par les programmes sont évidemment virtuelles (relatives) : une adresse comprend ici un numéro de page et un déplacement à l'intérieur de la page. Une ***table des pages*** permet de faire correspondre les pages virtuelles en mémoire virtuelle avec les pages réelles.

3.4.2 EXEMPLE

L'adresse virtuelle 12308 est située 20 octets après le début de la page virtuelle n°4 (adresses virtuelles 12288 à 16384) et correspond à l'adresse physique 40960 + 20 (page réelle n°11) soit 40980.

La MMU recalcule l'adresse physique de la donnée à déplacer en utilisant la table des pages. (voir schéma précédent).

Nota : Une adresse virtuelle se calcule en deux temps : adresse virtuelle = N° de segment + N° de la page dans le segment + déplacement dans la page.

Si un programme tente d'accéder à une page non présente en mémoire centrale, le système sauvegarde alors une page "bien choisie" sur le disque pour la remplacer par la page nécessaire (comme un segment pour la segmentation).

La pagination, contrairement à la segmentation, n'est pas connue du programmeur, elle est entièrement gérée par le système d'exploitation.

En effet, il serait absurde de découper un programme quelconque en sous-programmes de taille identique. Ainsi, la pagination n'oblige pas le programmeur à découper son programme.

En réalité, le système chargera tout ou partie d'un sous-programme ou du programme.

Une procédure pourra tenir en une ou plusieurs pages mais la dernière chargée sera probablement presque vide. Une des conséquences de la pagination est la présence en mémoire centrale de blocs non remplis, d'où une perte de place. Mais cet inconvénient est minime par rapport à l'intérêt de la pagination qui permet de charger un programme quelle que soit sa taille, même si des les blocs sont mal utilisés.

3.4.3 CHOIX DE LA TAILLE DES PAGES

Les principaux facteurs qui conditionnent le choix de la taille des pages sont :

- La fragmentation interne,
- L'espace occupé par les tables de description,
- Les entrées-sorties sur le disque,
- La structure des programmes.

3.4.3.1 FRAGMENTATION INTERNE

Il s'agit, en fait, de la perte de place due au remplissage incomplet de la dernière page d'un programme. Cette perte varie de 0 (cas idéal) à la taille d'une page moins un octet (le pire des cas). Si la taille T des pages est très inférieure à celle des programmes, la perte moyenne est de 1/2T par programme. Plus T est petit, plus la perte est faible.

3.4.3.2 ESPACE OCCUPE PAR LES TABLES

De toute évidence, plus les pages sont petites, plus il y aura d'entrées dans les tables de pages. L'espace occupé par les tables en mémoire centrale sera donc plus important.

3.4.3.3 ENTREES-SORTIES SUR LE DISQUE

Comme on a déjà vu, plus les pages sont petites, plus elles sont nombreuses. On aura donc plus d'entrées-sorties consommatrices de temps processeur. Si l'on s'en tient uniquement à ce facteur, il est intéressant d'avoir de grandes pages (nota : durée d'une e/s = temps de positionnement des têtes de lecture + temps de transfert).

3.4.3.4 STRUCTURE DES PROGRAMMES

L'examen de la structure des programmes montre qu'il y a systématiquement des zones plus ou moins actives, des zones qui seront très rarement exécutées. On peut donc penser que plus les pages seront petites, plus on aura de chances d'isoler sur des pages distinctes le code peu ou pas utilisé.

3.5 STRATEGIES DE REMPLACEMENT

3.5.1 REMPLACEMENT DES SEGMENTS

Deux méthodes sont utilisées :

3.5.1.1 LE PREMIER QUI CONVIENT (FIRST FIT)

Le premier espace dont la taille est supérieure à l'espace demandé est alloué au processus. Cette technique est très simple à implémenter mais occasionne de la perte de place. En effet, si le besoin est de 20 Ko et que le premier trou que le système trouve fait 60 Ko, il allouera cette place au segment.

3.5.1.2 CELUI QUI CONVIENT LE MIEUX (BEST FIT)

Le système devra gérer tout l'espace libre afin d'attribuer la zone libre dont la taille est la plus proche de la taille requise pour le segment. Cette technique est la plus coûteuse en gestion mais elle est la meilleure en efficacité même si, au bout d'un certain temps, on se retrouve avec beaucoup de zones libres trop petites. Bien entendu, il faudra exploiter la méthode de swapping décrite précédemment lors de l'expulsion de segments vers la mémoire auxiliaire

Après avoir opté pour une méthode, il existe encore de nombreux choix :
- Si un segment est libéré près d'une zone vide, regroupe-t-on les trous ou non ?
- Pratique-t-on ou non le retassage ?

3.5.2 REMPLACEMENT DES PAGES

Le problème est de savoir quelle page remplacer pour libérer de la place pour la page à mettre en place. Les algorithmes sont nombreux et ne sont pas sans incidence sur la rapidité du système.

3.5.2.1 ALEATOIRE (RANDOM)

La libération de page est aléatoire.

3.5.2.2 FIFO (FIRST IN FIRST OUT OU PEPS, PREMIER ENTRE PREMIER SORTI)

On expulse la page qui fut la première chargée en mémoire. L'algorithme est simple à mettre en œuvre, l'inconvénient est qu'il fonctionne très mal quand le système devient chargé, la page la plus vieille n'est pas nécessairement la moins utilisée. Bien qu'attirant par sa simplicité, il est assez peu utilisé.

3.5.2.3 LRU (LEAST RECENTLY USED OU MRU, MOINS RECEMMENT UTILISE)

On expulse la page dont l'utilisation est la plus reculée dans le temps. Ici, il sera nécessaire d'adjoindre un compteur à chaque page ce qui implique un coût élevé en gestion et en place mémoire. Il s'agit cependant d'un algorithme souvent utilisé car très satisfaisant par ses résultats.

3.5.2.4 LFU (LEAST FREQUENTLY USED OU MFU, MOINS FREQUEMMENT UTILISE)

On expulse la page la moins souvent utilisée pendant un intervalle de temps déterminé. Il faudra à nouveau un compteur. L'inconvénient majeur est que la page la moins utilisée jusqu'à présent n'est pas forcément celle qui le sera le moins dans un futur proche. En effet, si la page a été peu utilisée mais que ce ''peu'' a été fait récemment, il y a des chances pour qu'elle soit à nouveau appelée sous peu. Cette technique est très peu utilisée.

3.5.2.5 MIXTE (OPT POUR OPTIMISED)

Il s'agit d'un mélange des algorithmes LFU et LRU. L'expérience a montré que cette méthode ne présentait pas d'avantages décisifs et qu'elle était très coûteuse en place mémoire et en implémentation. Elle est très peu utilisée dans ce cadre là.

✸✸✸

Dans la pratique, les systèmes actuels combinent les principes de pagination et de segmentation :

- Les segments sont divisés en pages de taille fixe.
- Les pages de taille fixe sont divisées en segments de taille variable.
- Pagination à deux niveaux (hyperpage).

4. LA GESTION DES TRAVAUX (PROCESSUS) ET DES RESSOURCES

4.1 DEFINITIONS

4.1.1 PROCESSEUR

Un ***processeur*** est une ***unité élémentaire de traitement***, qui peut se présenter sous la forme d'un circuit (microprocesseur) ou d'un ensemble de circuits (cas des mini-ordinateurs et des gros systèmes).

4.1.2 PROCESSUS

Un ***processus*** est un ***programme en cours d'exécution***, l'action d'exécution d'un programme par un processeur.

Une analogie pour distinguer programme et processus : le processus est la suite de mots prononcés par un acteur, tandis que le programme est le texte sur papier de cette suite de mots. A chaque processus est associé un vecteur d'état qui donne l'ensemble des informations nécessaires à l'exécution du programme (registres, compteur programme, ...)

Un processus peut prendre différents états que nous évoquerons plus loin.

4.2 MODES DE FONCTIONNEMENT (RAPPEL)

4.2.1 MONOPROGRAMMATION

Un seul programme est exécuté à la fois, par un seul processeur. La notion de processus ne s'applique donc pas vraiment ici, le processus étant toujours actif.

4.2.2 MULTIPROGRAMMATION

Plusieurs processus sont présents à un moment donné, et sont activés un par un, ce qui permet de donner l'impression que l'utilisateur lambda est seul à travailler sur l'ordinateur. C'est le cas que nous étudierons dans les paragraphes suivants.

4.2.3 MULTITRAITEMENT

Il correspond au fonctionnement des ordinateurs à plusieurs processeurs : chaque processeur est occupé par un processus, de telle manière à optimiser les temps d'exécution.

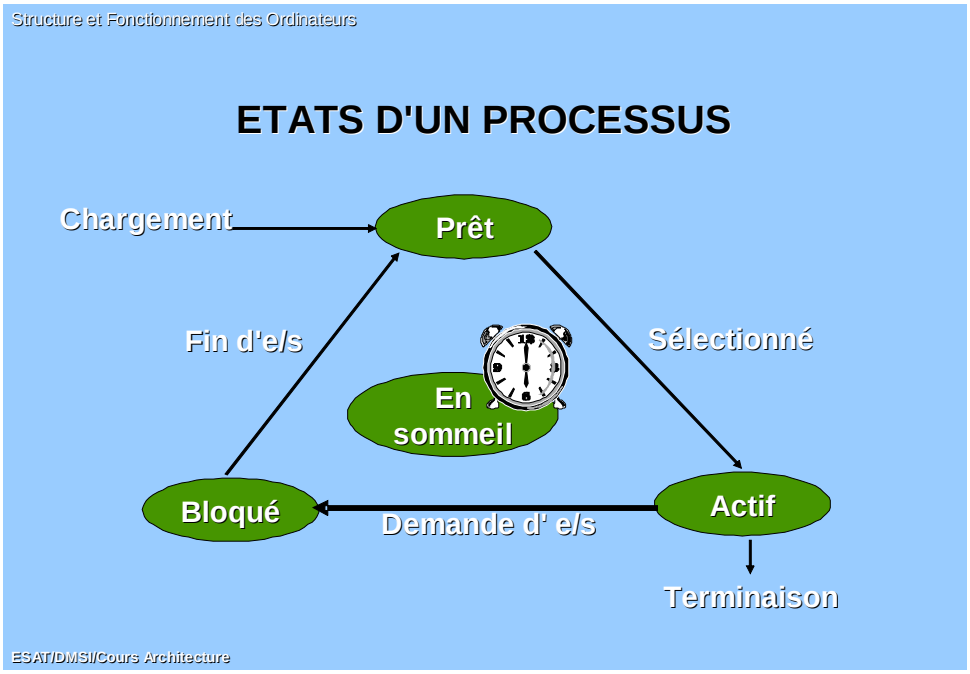
Par la suite, nous n'étudierons que la multiprogrammation.

4.3 ETAT DES PROCESSUS

A un instant donné, un processus peut-être existant ou inexistant :

- I***nexistant*** : soit il n'a jamais existé (***non existant***), soit il n'existe plus (***terminé***).

- Un processus ***existant*** pourra prendre les 4 principaux états suivant :
  - ***actif*** : en train d'être déroulé par le processeur (***élu)*** ;
  - ***prêt*** : en attente d'être déroulé par le processeur (***éligible***) ;
  - ***bloqué*** : en attente d'un signal en provenance d'une unité d'échange ou d'un autre processus ;
  - ***en sommeil*** : il attend un laps de temps donné avant de s'activer.



4.4 ALLOCATION DE L'UNITE CENTRALE

4.4.1 NOTION D'ORDONNANCEMENT (SCHEDULING)

Dans un système en multiprogrammation, plusieurs programmes doivent être exécutés concurremment par un processeur. Il est donc nécessaire de mettre en place une gestion de l'attribution du processeur aux différents processus représentant ces programmes. Cette politique de gestion s'appelle l'ordonnancement.

4.4.2 NOTION DE PRIORITE

Pour déterminer l'ordre de passage des différents processus, en fonction de leur importance, le système affecte à chacun un niveau de priorité. Ainsi, lorsque deux processus demandent le contrôle de l'unité centrale en même temps, le système d'exploitation privilégie celui qui a le niveau de priorité le plus élevé.

Problème : Si le processus de priorité maximal prend le contrôle de l'unité centrale, il ne la libère que pour faire des entrées-sorties, ou lorsqu'il a terminé son exécution, donc les autres processus ne peuvent pas travailler. Pour cela on introduit la notion de *quantum*.

4.4.3 NOTION DE QUANTUM

Définition : temps maximum pendant lequel un processus garde le contrôle de l'unité centrale sans être interrompu.

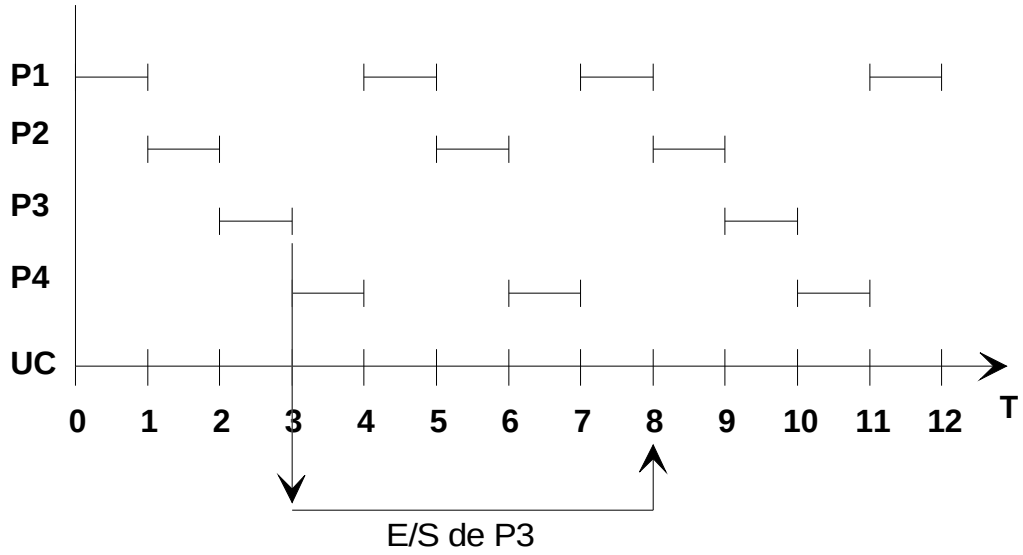
Passé ce temps, le processus est obligé de s'interrompre pour donner le contrôle à un autre processus, même s'il est moins prioritaire. La taille du quanta dépend du niveau de priorité du processus (pour les systèmes High Priority First) ou bien en est indépendante (dans les systèmes Round Robin).

4.4.4 DIFFERENTES STRATEGIES D'ORDONNACEMENT

4.4.4.1 MANEGE OU TOURNIQUET (ROUND ROBIN : RR)

Cette stratégie consiste à utiliser une file d'attente sans tenir compte des priorités Le premier processus de la file prend le contrôle du processeur pendant un quantum T, puis il est placé en queue de la file d'attente, le suivant s'exécute, et ainsi de suite. Si le processus dont c'est le tour ne peut-être activé (attente d'entrée-sortie par exemple), l'ordonnanceur passe au suivant.

Exemple :



P3 commande à 3T une E/S qui dure 5T, l'ordonnanceur, à 6T, débloque P4 puisque P3 est bloqué

Ordonnancement en tourniquet

4.4.4.2 PLUS HAUTE PRIORITÉ D'ABORD (HIGHEST PRIORITY FIRST : HPF)

Les processus sont rangés par ordre de priorité croissante dans la liste des processus prêts. Quand le système donne le contrôle du processeur à un processus, il prend le premier de la liste. Après expiration des quantum, le processus est remis dans la liste, puis rangé en fonction de son niveau de priorité (même chose s'il est bloqué en attente d'entrées-sorties).

Priorité aux processus courts : la priorité est inversement proportionnelle au temps d'exécution estimé du processus ; efficace si la majorité des processus sont courts.

Priorité aux processus effectuant des entrées-sorties : on favorise les processus qui ont des temps d'attente de façon à diminuer le temps de réponse.

Priorité variable au cours du temps : la priorité associée à un processus augmente en fonction de son état d'avancement.

Exemple : quatre processus (P1, P2, P3 et P4), quantum T

P1 : processus effectuant des E/S, l'envoi d'un message dur T/2 et la réponse arrive au bout de 2T.

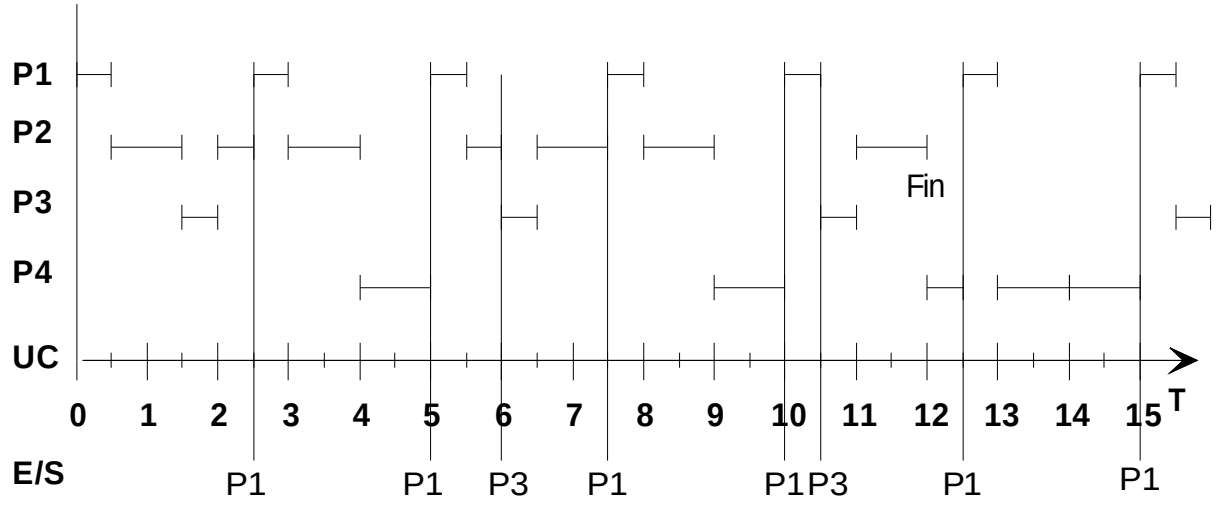
P2 : processus de calcul de durée 6T.

P3 : processus effectuant des E/S, l'envoi d'un message dur T/2 et la réponse arrive au bout de 4T.

P4 : processus de calcul de durée 9T.

Priorités : P1>P2>P3>P4.

Algorithme : HPF avec priorité aux processus effectuant des E/S.


P1
P2
P3
P4
UC
Fin
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
T
E/S
P1
P1
P3
P1
P1P3
P1
P1

5. LA GESTION DE L'ENVIRONNEMENT ET DES ERREURS

Ce chapitre expose d'une manière générale les principes du dialogue homme-machine, de la gestion des disques (du point de vue de l'utilisateur) et de la gestion des erreurs.

5.1 L'INTERPRETEUR DE COMMANDES (SHELL : ENVIRONNEMENT DE TRAVAIL)

5.1.1 ROLE ET FONCTIONS

L'interpréteur de commandes permet à l'utilisateur de dialoguer avec l'ordinateur grâce à un ensemble de commandes. Pour ce faire, il :

- Vérifie la syntaxe de la commande frappée (fautes de frappes, oublis),
- Vérifie la conformité des paramètres qui suivent cette commande,
- Exécute la commande si les deux points précédents sont corrects, sinon retourne à l'utilisateur un message adapté.

5.1.2 LES DEUX TYPES DE COMMANDES

- Les ***commandes internes*** : elles sont incluses dans le Shell (interpréteur de commandes) et donc résidentes en mémoire dès que celui-ci est chargé. Par conséquent, aucun accès disque n'est nécessaire pour les exécuter.

- Les ***commandes externes*** : elles sont répertoriées dans une bibliothèque système sur disque (disquette ou disque dur) et sont chargées en RAM dès qu'il est fait appel. Elles sont donc traitées comme n'importe quel programme exécutable. Certaines commandes externes sont tellement évoluées que l'on parle davantage d'***utilitaires*** apparentés à des outils applicatifs spécifiques.

Pourquoi deux types de commandes ?

Explication : la RAM des ordinateurs étant limitée, on ne conserve en mémoire que les commandes les plus utiles (les plus souvent exécutées). Toute place prise en RAM par le système d'exploitation est, en effet, autant de place disponible en moins pour les programmes d'application. On trouve en général en commandes internes tout ce qui concerne la gestion simple du disque (catalogue, copie, effacement et renommage de fichiers, gestion des sous-répertoires, etc.).

5.1.3 PRINCIPES DU DIALOGUE

Le système d'exploitation indique à l'utilisateur qu'il est prêt à recevoir une commande en affichant un message d'invite (prompt). Ce prompt comprend généralement un caractère (> pour MS/DOS, $ pour UNIX) précédé ou non d'informations définies par l'utilisateur (répertoire courant, date, heure, nom de l'utilisateur, etc.). Les commandes frappées viennent s'inscrire derrière cette invite.

5.2 LA GESTION DE L'ESPACE DISQUE

C'est une notion très importante en raison de l'augmentation de la capacité des disques durs : il est indispensable de pouvoir mettre de l'ordre dans les milliers de fichiers qui peuvent se trouver sur un disque. Ce rangement est effectué au moyen de sous-répertoires.

5.2.1 NOTION DE FICHIER

Définition : un fichier est une collection organisée d'informations présentant entre elles un lien logique. Ces informations peuvent constituer un programme exécutable ou un ensemble de données quelconques.

Un fichier est identifié et reconnu du système d'exploitation par son :

- Nom : un nom de fichier comprend 8 caractères sous MS/DOS.

- Extension : Sous MS/DOS, un ensemble de 3 caractères séparés du nom par un point constitue l'extension. Cette extension sert à repérer les fichiers par type. Exemple de dénomination de fichiers : CT2.LST, SOURCE.PAS...

Extensions courantes de MS/DOS : .COM et .EXE (programmes exécutables), .SYS (fichiers systèmes), .BAT (fichiers de commandes), .PAS (sources pascal), .BAK (fichiers de sauvegarde des anciennes versions).

5.2.2 NOTION D'UNITE LOGIQUE

Le système d'exploitation doit pouvoir reconnaître le support physique (disquette, disque dur) sur lequel il est en train de travailler. Il attribue donc à ces supports une ou plusieurs unités logiques (lettres sous MS/DOS) permettant de les identifier :

- A : pour un lecteur de disquettes,
- C : pour un disque dur,
- D : pour un lecteur cédérom.

Remarque : Si la capacité d'un disque dur est élevée, il peut être découpé (partitionné) en plusieurs unités logiques afin d'en faciliter la gestion.

5.2.3 NOTION DE SOUS-REPERTOIRE

Un disque dur moyen possède de nos jours une capacité de 300 Mo à 2,4 Go. Or une disquette peut contenir au maximum 2,88 Mo (1,44 Mo pour les P.C.) : on voit donc qu'un disque dur peut contenir des milliers de fichiers et qu'il est nécessaire de classer tous ces fichiers. On va utiliser des sous-répertoires (ou dossiers par analogie avec les systèmes de classement).

Règle à respecter : Eviter de mélanger les fichiers provenant d'applications différentes, donc grouper indépendamment fichiers système, fichiers PASCAL, fichiers traitement de texte..., séparer les utilisateurs entre eux dans le cas d'un système multi-utilisateur (UNIX).

On peut faire l'analogie avec une armoire de rangement :

- Armoire : disque dur,
- Tiroirs : sous-répertoire de premier niveau,
- Classeurs : sous-répertoire de deuxième niveau,
- Feuille dans le classeur : fichiers.

Chaque niveau de sous-répertoire peut donc contenir des fichiers ou des sous-répertoires. Ceci permet donc de créer des groupes de fichiers distincts. Sous MS/DOS, le répertoire racine contient les fichiers du système d'exploitation (on peut par contre ranger les commandes externes dans un sous-répertoire).

Voici un exemple d'arborescence d'une partie de disque :

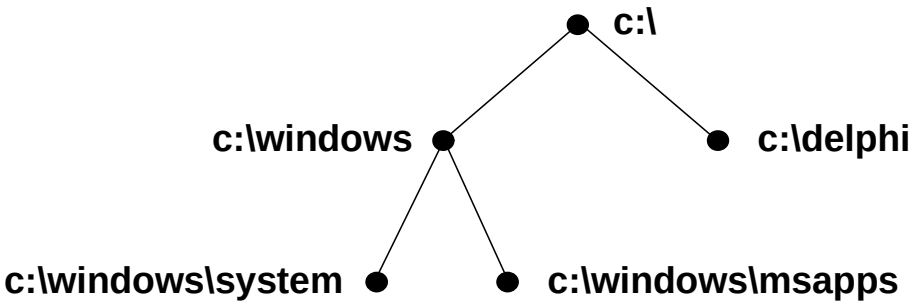


Le ***répertoire courant*** est celui sous lequel l'ordinateur travaille à un instant donné.

5.2.4 NOTION DE CHEMIN D'ACCES A UN FICHIER

Le chemin d'accès à un fichier (path) est constitué de la liste des sous-répertoires qu'il faut traverser pour accéder à ce fichier à partir de la racine.

Arborescence d'un disque dur :



5.3 GESTION DES INCIDENTS

Un ordinateur est un appareil sensible, et la moindre défaillance peut parfois entraîner des incidents graves.

Ces incidents peuvent aller de la simple erreur de programmation, jusqu'à la coupure de courant. Le système d'exploitation doit alors intervenir afin de minimiser les risques de pertes de données, en détectant et en diagnostiquant les incidents le plus rapidement possible.

5.3.1 PRINCIPAUX INCIDENTS

- Erreurs de syntaxe : mauvaise formulation d'une commande, détectée par le shell.

- Erreurs d'exécution : débordement mémoire, accès à une zone mémoire réservée ou interdite, division par zéro...

- Problèmes de fichiers : fichiers introuvables, fichiers protégés, supports pleins...

- **Problèmes de sous-répertoires** : création d'un sous-répertoire déjà existant, effacement d'un sous-répertoire non vide, sous-répertoire inexistant...

- **Problèmes d'entrées-sorties** : disquette absente, défectueuse ou protégée, imprimante non prête, problème de liaison avec un périphérique...

- **Problèmes matériels** : panne électronique, panne de périphériques, surchauffe des composants (ventilation interne insuffisante), ...

- **Problèmes indépendants du matériel** : panne d'électricité, surchauffe du système (ventilation du local insuffisante)...

5.3.2 REACTION AUX INCIDENTS

La prise en compte des incidents est effectuée soit par l'intermédiaire d'un code d'erreur retourné par la fonction appelée, soit par une interruption lorsque le problème est plus sérieux (problème d'accès mémoire, division par zéro). Le système exécute alors une routine de prise en compte de l'incident et affiche, dans la plupart des cas, un message d'erreur.

5.3.3 MAINTENANCE DES MEMOIRES DE MASSE

La capacité des mémoires de masse augmentant sans cesse, la perte d'informations présentes sur les disques durs peut représenter un préjudice très important pour un organisme (tant au niveau du contenu que du temps de travail). Il est par conséquent indispensable de pouvoir à tout moment (ou presque...) disposer d'un jeu de fichiers contenant l'état du ou des disques durs à un instant donné. Ceci peut être obtenu soit par sauvegarde soit par archivage.

5.3.3.1 SAUVEGARDE (BACKUP)

Elle consiste à copier l'ensemble des fichiers contenus sur le ou les disques durs, sur un autre support magnétique (disquettes ou bandes), à l'exception des logiciels du commerce.

Il est indispensable d'utiliser deux jeux de sauvegarde et de faire la sauvegarde de façon régulière. En effet, des données pourraient être endommagées sur le disque dur **et** sur les copies de sauvegarde en cas de problème, d'où l'utilité du second jeu. La fréquence des sauvegardes doit être proportionnelle à la quantité de travail effectuée : quotidienne, hebdomadaire, mensuelle.

5.3.3.2 ARCHIVAGE

Il consiste à copier certains fichiers du ou des disques durs (concernant généralement une application donnée) sur disquette ou bande, cette fois dans l'intention de faire de la place. Les supports seront conservés et utilisés en cas de besoin.

6. LA GESTION DES FICHIERS

Tout système d'exploitation doit posséder des outils de gestion de fichiers : MS-DOS possède un système de gestion de fichiers propre, Unix utilise un File System particulier, Windows NT utilise NTFS (NT File System) étudié pour ce système.

Le système de gestion de fichiers d'un ordinateur est responsable de l'organisation et de l'accès aux informations.

Pour l'utilisateur, un fichier est une identité de dimension arbitraire qui forme un tout. Pour le système de gestion de fichier, un fichier est une unité logique qu'il doit pouvoir gérer en fonction des espaces physiques disponibles sur les supports périphériques.

Un système de gestion de fichiers doit :

- Permettre la création et la suppression de fichiers.
- Permettre l'accès au fichier en lecture et en écriture.
- Gérer automatiquement les mémoires de masse dont l'utilisateur ne doit pas s'occuper.
- Permettre d'appeler les fichiers par un nom symbolique que manipulera l'utilisateur.
- Protéger les fichiers contre les défaillances du système de façon à assurer leur intégrité.
- Permettre le partage de fichiers entre utilisateurs tout en les protégeant contre les accès non autorisés.

Pour arriver à réaliser ces fonctions, le système de gestion de fichier, partie intégrante du système d'exploitation, dispose au moins d'une table de correspondance (répertoire, catalogue) qui fait le lien entre le nom symbolique du fichier et les caractéristiques physiques du fichier sur la machine qui sont :

- Unité où il est implanté,
- Taille,
- Type du fichier,
- Eléments de contrôle d'accès,
- Date et heure de dernière mise à jour.

L'ouverture d'un fichier est une procédure qui permet au système d'exploitation d'assurer le flot d'entrée-sortie en contrôlant l'unité périphérique et le support de stockage du fichier.

Afin de donner des notions simples, le système de gestion de fichiers sera abordé au travers de l'exemple d'une disquette de 3,5'' formatée sous MS-DOS.

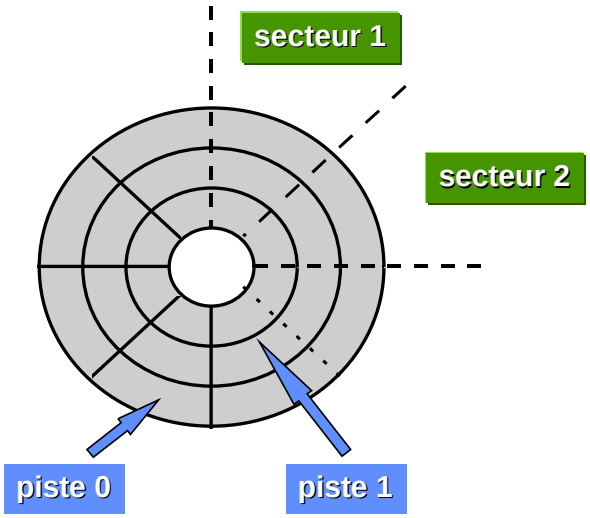
6.1 LE SYSTEME DE GESTION DE FICHIERS DE MS-DOS SUR UNE DISQUETTE 3,5''

6.1.1 LE SYSTEME DE GESTION DE FICHIERS

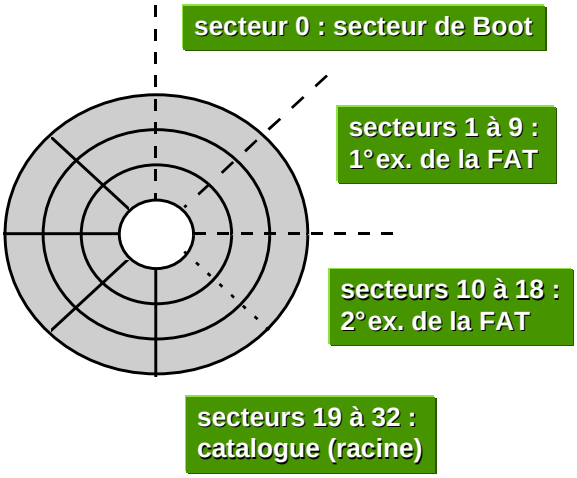
Dans ce qui suit, le système de gestion de fichiers sera étudié en prenant comme exemple une disquette d'une capacité de 1,44 MO.

Avant de se servir d'un disque dur ou d'une disquette il est nécessaire d'effectuer un formatage physique puis un formatage logique.

Le ***formatage physique*** s'effectue en atelier chez le constructeur on appelle aussi ce formatage le formatage de bas niveau. Il consiste à tracer les pistes magnétiques sur la surface des disques et à découper ces pistes en secteurs.



Le ***formatage logique*** consiste à installer le système de gestion de fichiers sur la disquette.



Une fois formatée, nous trouvons sur la disquette :

- Un ***secteur de boot*** (premier secteur du support : secteur 0) qui permet de repérer s'il s'agit d'une disquette système ou d'un disque dur "bootable" (auto - démarrage). Dans ce cas, la petite routine qui s'y trouve charge les fichiers de MS/DOS. Si la disquette n'est pas une disquette système, on retrouve les caractéristiques de la disquette et un message d'erreur.

- la ***FAT*** (File Allocation Table : table d'allocation des fichiers) qui contient la liste des secteurs du disque utilisés par chaque fichier (secteurs 1 à 9).

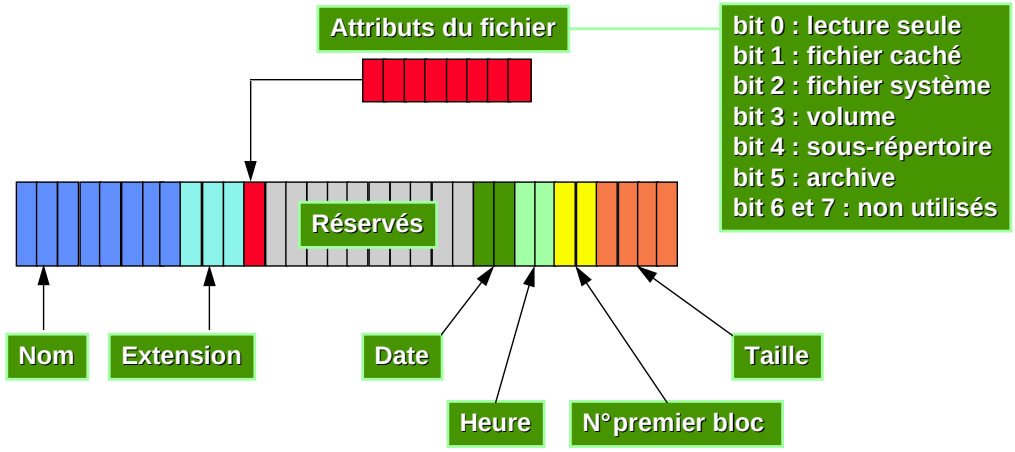
- une ***copie*** de sûreté de la FAT (secteurs 10 à 18).

- un ***catalogue*** (directory) qui contient la liste des fichiers et leurs attributs (caché, système, lecture seule...) ainsi que des sous-répertoires (secteurs 19 à 32).

Une disquette 1,44 MO contient 2880 secteurs il reste donc 2880-32 = 2848 secteurs pour les utilisateurs.

6.1.2 COMPOSITION DU REPERTOIRE RACINE (DIRECTORY)

On y trouve des informations sur les fichiers. Certaines de ces informations sont accessibles sous MS-DOS par la commande DIR. Le répertoire racine est découpé en entrées de répertoire de 32 octets chacune comme l'illustre le schéma ci-dessous :



Pour une entrée de répertoire, on a la structure suivante :

Nom du fichier (nom + extension) :	11 octets,
Attributs du fichier :	1 octet,
Réservés :	10 octets,
Heure de création ou modification :	2 octets,
Date de création ou modification :	2 octets,
N° du premier bloc du fichier :	2 octets,
Taille du fichier :	4 octets.

Les bits de l'octet d'attributs de l'entrée de catalogue signifient :
Bit 0 positionné à 1 : l'entrée de catalogue est accessible seulement en lecture (modification impossible),
Bit 1 positionné à 1 : l'entrée de catalogue est une entrée cachée (invisible par la commande DIR),
Bit 2 positionné à 1 : l'entrée de catalogue est une entrée système,
Bit 3 positionné à 1 : l'entrée de catalogue décrit un volume,
Bit 4 positionné à 1 : l'entrée de catalogue décrit un sous-répertoire,
Bit 5 positionné à 1 : l'entrée de catalogue est une entrée archive,
Bit 6 non utilisé
Bit 7 non utilisé

Le répertoire racine occupe 14 secteurs (19 à 32) c'est à dire 14 X 512 octets = 7168 octets. Or, chaque entrée du catalogue fait 32 octets, on peut donc décrire 7168 / 32 = 224 fichiers ou sous répertoires.

Pour déclarer un sous répertoire, on le fait comme pour un fichier mais on met le bit 4 de l'octet attribut à 1 et on ne mentionne pas la taille du fichier. Sa structure est la même que celle du répertoire racine. Un sous-répertoire peut lui-même contenir d'autres sous-répertoires.

6.1.3 LA FAT

La FAT est une suite de mots de deux octets ; chaque mot est appelé "entrée de FAT" et fait donc 16 bits.

Sous MS-DOS tous les disques sont formatés par secteurs de 512 octets.

En outre, MS-DOS définit une unité d'allocation de ressource appelée ***cluster*** : cette unité est la quantité minimale de la ressource pouvant être attribuée à un programme sur le support. Dans le cas d'une disquette 1,44 MO le cluster représente 2 secteurs soit 1024 octets (1 Ko).

Quelle que soit la taille d'un fichier, celui-ci occupera un nombre entier de clusters.

Chaque entrée de la FAT contient des informations sur l'état du cluster considéré selon les codes suivants :

000 :	disponible,
FF0 - FF6 :	réservé au DOS,
FF7 :	détruit ou rayé de la FAT,
FF8 - FFF :	dernier cluster alloué à un fichier,
XXXX :	numéro relatif au prochain cluster alloué au fichier.

Ce système permet le chaînage des clusters alloués à un fichier, on remonte ainsi la chaîne jusqu'à la lecture de FF8-FFF.

6.1.4 LECTURE D'UN FICHIER SUR LA DISQUETTE

Dans un premier temps la FAT et le Directory de la disquette sont chargés en mémoire centrale pour lecture.

Le fichier est ensuite recherché dans le directory du disque grâce à son nom puis on lit dans son entrée quel est le premier cluster qui lui est alloué.

Ce cluster est alors lu et transféré en mémoire centrale, on récupère les données ou le code du fichier et ensuite on peut y lire le numéro du cluster suivant que l'on chargera en mémoire centrale et ainsi de suite jusqu'à ce qu'on arrive au cluster marqué FF8-FFF.

6.2 LA STRUCTURE DE L'INFORMATION

Quel que soit le type de traitement, il est nécessaire de pouvoir stocker sur des supports (disque, disquette, bande) des données et des programmes (d'exploitation, utilitaire ou d'application). On peut distinguer les fichiers résidents, ceux qui sont sur des mémoires auxiliaires, de ceux qui sont archivés à l'extérieur du système, sur bande, sur cartouches ou disquettes.

Les fichiers peuvent être des fichiers de données pour une application de gestion, des fichiers programmes ou des fichiers textes.

Les programmes d'exploitation et les utilitaires sont presque toujours des programmes résidents, alors que les programmes d'application et les fichiers de données ne sont chargés que selon le besoin.

Nous ne parlerons pas des fichiers type texte (lettres ou autres documents lisibles par un traitement de texte). Nous ne parlerons pas des fichiers exécutables dont le contenu est illisible par l'utilisateur. Seuls seront évoqués des fichiers destinés à décrire une réalité quelconque comme les abonnés de France Télécom, les articles d'un magasin, le personnel d'une administration, etc.

Ces fichiers contiennent deux types d'information : de l'alphabétique ou chaîne de caractère, ou du numérique (nombres réels ou entiers).

6.2.1 EXEMPLES

Le nom TOTO est une chaîne de 4 caractères.

L'adresse ''12 rue des Lilas'' est une chaîne de 16 caractères (l'espace étant aussi un caractère). Précisons que cette chaîne contient un nombre (12) qui n'est pas un nombre au sens strict : c'est un numéro à partir duquel on ne calculera rien (pas d'addition de numéro de rue, ni de statistiques !). Le nombre 12 est par conséquent une chaîne de deux caractères « 1 » et « 2 ».

6.2.2 CODAGE

Nous savons que les caractères (a b c ... A B C ... 1 2 3 ... + - * ! % µ @ ... _ ¦ ...) sont codés sur 1 octet, soit 8 bits. Avec 8 bits, nous codons 256 caractères différents. La lettre 'A' est codée 41 en Hexadécimal soit 00110001 en binaire. Le caractère '1' est codé 00100001.

Codons à présent les nombres. Si nous codons tous les caractères sur 1 octet, il ne nous reste plus aucune possibilité pour coder les nombres. Il faut un codage spécifique pour les nombres.

Si l'on veut coder des grandeurs suffisamment significatives, il faut dépasser la limite de l'octet (de 0 à 255 : c'est faible) et choisir un mot de 2 octets (de 0 à 65535 ou de -32768 à +32767). Pour dépasser ces valeurs, on utilise un codage spécial réservé aux nombres réels : codification des multiples de puissances de 10 (655 350 = 65,535 X 10^4. Nous ne considérerons ici que les entiers codés sur 2 octets. Le nombre 1 sera alors codé : 0000000000000001 (remarquez que cela n'a rien à voir avec le codage du caractère 1).

6.2.3 DESCRIPTION D'UNE REALITE : LE FICHIER

Décrivons à présent la réalité suivante : les personnels du service comptable. On souhaite connaître le nom, le département et l'âge des gens.

Exemple : TOTO LOT 28 signifie que TOTO est du LOT et qu'il a 28 ans.

En hexadécimal cela donne : **544F544F** 4C4F54 **001C**

TOTO LOT **28**

Ce qui donne en binaire :

01000011 **11110100** 01000011 **11110100** 00111100 **11110100** 01000011 **0000000000011100**

T **O** T **O** L **O** T **28**

De même, on code les autres personnels : DURAND GERS 35, DUPONT VIENNE 40...

A présent, un problème crucial se présente : comment lire le fichier ? Comment savoir dans ce flot d'octets que **544F544F**4C4F54**001C** signifie TOTO Lot 28 et non « T Otolo T 'espace' '¬' » '¬' étant un caractère codé par 1C et 'espace' le caractère codé 00 ? Comment savoir que 001C est le code de 28 sur 2 octets et non pas deux caractères chacun sur 1 octet ? Comment savoir quels sont les octets correspondant au nom, ceux au département,..., d'autant plus que tous les noms ou départements n'ont

pas le même nombre de lettres donc d'octets ! Quand commence l'information relative à TOTO, celle à Martin, etc. Où se termine le fichier ?

Il est impossible d'utiliser un fichier si vous ne savez pas la manière dont il est organisé. Il existe deux grands types d'organisation des fichiers : à structure fixe ou à structure variable.

6.3 ORGANISATION LOGIQUE A STRUCTURE FIXE

6.3.1 ORGANISATION LOGIQUE STRUCTUREE

Cette organisation consiste à créer un format, une ***structure*** pour le fichier avant même de commencer à stocker des informations. Le programmeur va décider par exemple, que le nom est codé sur 15 octets, et le département sur 20, quant à l'âge, il sera obligatoirement sur 2 octets (numérique). Pour TOTO, nous obtenons :

TOTO 11 octets libres LOT 17 octets libres **28**MARTIN....

Ainsi, si l'on souhaite lire le nom il faut lire les 15 premiers octets et c'est tout. Lorsqu'on a lu 37 octets (15+20+2), on sait qu'il s'agit alors d'un autre individu.

6.3.2 NOTION D'ENREGISTREMENT

Chaque individu est ainsi décrit par un nombre fixe d'octets (ici 37). Ce groupe d'octets constitue un ***enregistrement***. L'enregistrement est lui-même subdivisé sous-groupes d'octets appelés ***champs*** ou ***rubriques***. Dans notre exemple, nous considérons 3 champs : NOM, DEPARTEMENT, AGE. La notion d'enregistrement va de pair avec celle de numéro d'enregistrement. Ce numéro est rajouté à la structure afin de faciliter le travail de recherche, il correspond à l'emplacement des enregistrements les uns par rapport aux autres. Un enregistrement est stocké de manière contiguë en mémoire centrale.

N° d'enregistrement	NOM (15)	DEPARTEMENT (20)	AGE
1	TOTO.....	LOT.....	28
2	MARTIN.....	VIENNE.....	30
3	DUPONT.....	.....	

Notons que cette structure, bien que souvent rencontrée en gestion, est indépendante du fichier, elle fait partie du programme associé à l'exploitation du fichier (lecture ou écriture d'informations par ajout ou suppression d'enregistrements). En réalité, le fichier se présente comme nous l'avons vu, en une suite d'octets mis bout à bout. L'exploitation du fichier ne peut se faire sans aborder la notion d'accès.

6.3.3 ACCES A L'INFORMATION

Supposons que l'on souhaite accéder à un enregistrement quelconque d'un fichier en mémoire centrale. Il y a quatre manières d'y parvenir, selon le type d'organisation choisi.

6.3.3.1 ORGANISATION LOGIQUE SEQUENTIELLE CLASSIQUE

On accède à un enregistrement après avoir lu tous ceux qui le précèdent. Le fichier peut être trié pour aller plus vite. Un tri s'effectue en choisissant un critère de tri, le champ sur lequel on veut faire porter le classement. Exemple : tri sur le NOM : Durand, Martin, TOTO, Ying yang... Dans ce type d'organisation, le tri se caractérise par un déplacement physique des enregistrements.

Le tri permet d'arrêter une recherche lorsque le nom cherché n'est pas dans le groupe alphabétique correspondant. Exemple : La recherche de 'Titi' s'arrête lorsqu'on lit l'enregistrement 'TOTO' (To > Ti). Notons qu'un tri par déplacement physique des enregistrements est long. Il y a peu d'intérêt à utiliser ce style d'organisation lorsqu'un fichier est structuré en champs de tailles fixes.

6.3.3.2 ORGANISATION LOGIQUE SEQUENTIELLE CHAINEE

Les enregistrements peuvent être dans n'importe quel ordre et dispersés en mémoire. Une rubrique spéciale est rajoutée à la structure pour indiquer la position (adresse @) de l'enregistrement suivant :

Adresse	N° enreg	NOM (15)	DEPARTEMENT (20)	AGE	Suivant
@1	**3**	TOTO.....	LOT.....	28	@4
@2	**1**	DUPONT	VIENNE.....	30	@3
@3	**2**	MARTIN.....	CHARENTE	23	@1
@4	**4**	Ying Yang	....	..	@....

L'adresse indique la position physique des enregistrements en mémoire et le numéro d'enregistrement correspond à l'ordre de lecture du fichier. Ici, le numéro d'enregistrement n'a plus rien à voir avec la position réelle des enregistrements.

Les champs *NOM*, *DEPARTEMENT* et *AGE* appartiennent à la structure proprement dite, connue de l'utilisateur. Les autres informations (*Adresse*, *N° enreg*, *Suivant*) ne concernent que le programmeur. La recherche s'effectue en parcourant le fichier selon les numéros d'enregistrements et les adresses des suivants. Selon cette organisation, un tri s'effectue par simple modification des ***pointeurs*** de Suivant (dernière colonne du tableau) et des numéros d'enregistrement : **le fichier ne bouge pas** (gain de place et de temps).

6.3.3.3 ORGANISATION LOGIQUE INDEXEE

Cette organisation demande la création d'un fichier ***index*** trié qui contient le champ sur lequel porte la recherche et l'adresse de l'enregistrement, comme suit :

N°	NOM (trié)	ADRESSE
1	DUPONT	@2
2	MARTIN	@3
3	TOTO	@1
4	YING YANG	@4
...	.....	

Cet index s'associe au fichier décrit précédemment. Il n'est plus nécessaire de maintenir les rubriques *N° enreg* et *Suivant* dans le fichier principal. La recherche dans l'index est instantanée car directe. L'accès direct à un champ s'effectue grâce à un calcul (par exemple la ***dichotomie***). L'index donne l'adresse du premier octet de l'enregistrement ; ensuite, on accède directement à l'enregistrement cherché dans le fichier : rapidité d'accès.

Complément : recherche par ***dichotomie***.

Soit un fichier de 50 enregistrements. A l'aide de l'index sur le champ NOM, on va rechercher TOTO.
On lit le nom dont le numéro correspond à la moitié du nombre d'enregistrements (50 / 2 = 25).
Si le nom correspondant au numéro 25 est supérieur à TOTO alors on lit le nom dont le numéro correspond à la moitié de 25 (25 / 2 = 12,5 arrondi à 13).
Si le nom correspondant au numéro 13 est inférieur à TOTO alors on lit le nom correspondant au numéro [13+[(25-13)/2]], ainsi de suite jusqu'à ce que TOTO soit trouvé.
Les numéros 25, 13, ... sont des ***numéros pivots***.
Par inférieur ou supérieur, on entend une comparaison caractère par caractère des valeurs des codes ASCII (A = 41h, B = 42h, A < B).

6.3.3.4 INCONVENIENTS DE L'ORGANISATION LOGIQUE A STRUCTURE FIXE

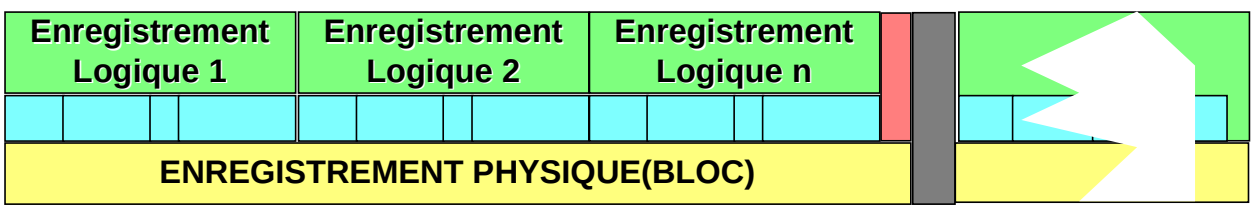
La clarté et l'efficacité d'une telle organisation se heurtent à un problème de gaspillage d'espace mémoire. En effet, la taille des champs ne peut être qu'approximative, quelque fois de même taille que son contenu mais très souvent de taille supérieure ; ce qui oblige à conserver des octets non significatifs en mémoire centrale et auxiliaire. La solution réside dans des enregistrements de tailles variables.

6.4 ORGANISATION LOGIQUE A STRUCTURE VARIABLE

Les champs de chaque enregistrement ont une taille qui correspond à leur contenu, à l'exception des champs numériques, qui par définition, restent codés sur 2 mots machine (sous DOS : 2 octets ; sous UNIX : 4 octets). Entre chaque champ, un octet est occupé par un séparateur ; il en est de même entre chaque enregistrement. Par convention, le caractère '\0' (00 en héxa) sépare les champs et '\n' (13 en héxa) marque la fin des enregistrements.

Cette structure permet tous les types d'accès vus précédemment mais sa gestion est plus rigoureuse. L'oubli d'un séparateur crée rapidement un parfait désordre. Cependant, le gain de place n'est pas négligeable : le fichier sera plus ''léger''.

6.5 ENREGISTREMENT LOGIQUE, ENREGISTREMENT PHYSIQUE



6.5.1 ENREGISTREMENT LOGIQUE

Les enregistrements (ou articles) groupent les informations élémentaires (octets). L'enregistrement logique est ***l'unité de base directement accessible par le programme utilisateur***.

Comme nous l'avons vu, le format d'un enregistrement peut-être :

- **Fixe** : tous les enregistrements ont la même longueur.

Ou

- **Variable** : la longueur des enregistrements est variable et précisée sous une rubrique au début de l'enregistrement.

6.5.2 ENREGISTREMENT PHYSIQUE

L'enregistrement physique est ***l'ensemble des caractères transférés par le système d'exploitation au cours d'une entrée-sortie*** avec le support. Un enregistrement physique a toujours une ***taille fixe*** : c'est la plus petite unité adressable sur le support.

Remarque : le système effectue ses entrées-sorties indépendamment de celles effectuées par le programmeur ; il attend le remplissage d'un tampon pour effectuer une écriture ou remplit un tampon complet en lecture. **Le système d'exploitation a donc pour rôle de faire la correspondance entre enregistrements physiques et enregistrements logiques**.

6.6 ORGANISATION PHYSIQUE

On distingue là encore une organisation séquentielle et une organisation adressée. Ces notions sont totalement indépendantes de l'organisation logique, elles concernent l'emplacement des enregistrements physiques sur le support.

6.6.1 ORGANISATION PHYSIQUE SEQUENTIELLE

C'est le cas des bandes magnétiques où tous les enregistrements physiques se suivent. La seule organisation logique possible est l'organisation séquentielle classique : on défile la bande depuis le début du fichier jusqu'à l'enregistrement cherché, ce qui entraîne une recherche parfois très longue. De nos jours, les bandes ne sont plus utilisées que pour la sauvegarde ou l'archivage.

6.6.2 ORGANISATION PHYSIQUE ADRESSEE

Cette organisation concerne essentiellement les disques (disquettes, disques durs, CD-ROM), où la tête de lecture peut se positionner directement à n'importe quel endroit. Le gain de temps par rapport à l'organisation séquentielle est donc important.

De plus, les fichiers peuvent être découpés en plusieurs parties logiquement chaînées, mais physiquement situées à des endroits différents du support (fragmentation). Ceci permet d'utiliser au mieux la capacité du support, mais augmente le temps de recherche en raison des mouvements de la tête de lecture. Il existe des utilitaires permettant de défragmenter le disque, c'est-à-dire de chaîner les enregistrements physiques afin que l'ensemble du fichier puisse être lu sans trop déplacer la tête de lecture ou bien, le système d'exploitation lui-même assure en permanence cette optimisation du support.

La liste des enregistrements physiques utilisés par un fichier est contenue dans la FAT (File Allocation Table) sous MS-DOS. Le premier mot de repérage dans la FAT est contenu dans le

répertoire du fichier. Cette organisation adressée permet d'implémenter tous les modes d'organisation logique évoqués précédemment.

6.7 EN RESUME

De façon classique on définit un fichier comme une collection d'objets identiques. Il existe plusieurs types de fichiers : séquentiel, séquentiel indexé, à accès direct.

L'organisation d'un ***fichier séquentiel*** est semblable à celle d'une suite de fiches dans un bac : les enregistrements se suivent les uns les autres sans aucun repère ; seul l'ordre des enregistrements peut compter. Quand on recherche un enregistrement, il faut lire tous ceux qui le précédent jusqu'à l'avoir trouvé. Un fichier séquentiel peut être stocké soit sur un ***support à accès séquentiel*** soit sur un ***support à accès direct***.

L'organisation d'un ***fichier à accès direct*** permet d'accéder à un enregistrement sans procéder à une recherche, en utilisant son adresse physique d'implantation sur le support. Différentes techniques sont utilisées pour gérer des fichiers à accès direct. Par exemple, on dispose d'une table de correspondance qui donne pour chaque clé (numéro) d'enregistrement l'adresse physique correspondante ; la table est stockée sur le support en même temps que le fichier et l'accès à un enregistrement se fait en deux temps : accès à la table pour connaître l'adresse de l'enregistrement, puis accès à l'enregistrement ; l'inconvénient de cette méthode tient au volume de la table dans le cas de gros fichiers. Un fichier à accès direct réside ***nécessairement*** sur un ***support à accès direct***.

L'organisation d'un fichier ***séquentiel indexé*** est à mi-chemin entre les deux organisations déjà vues. On définit une table des adresses d'implantation physique pour un nombre restreint d'enregistrements, dont la valeur de la clé est à intervalles réguliers et auxquels on peut donc accéder directement ; les enregistrements, dont les clés sont comprises entre deux clés d'enregistrement appartenant à cette table, sont rangés séquentiellement sur le support, entre deux enregistrements indexés formant les bornes supérieures et inférieures. L'accès à un enregistrement quelconque se fait en cherchant dans la table à quel intervalle de clé il appartient, puis en procédant à une recherche séquentielle sur le support à partir de la borne inférieure obtenue. Un fichier séquentiel indexé est ***nécessairement*** sur un ***support à accès direct***.

7. LA GESTION DES PROGRAMMES

Dès le début de l'informatique, il a paru difficile de programmer en langage machine interne. Très vite les constructeurs ont conçu et réalisé des programmes de traduction permettant de travailler dans des langages plus simples.

Les langages d'assemblage ont d'abord été créés, encore très proches de la structure d'un ordinateur. Puis on a imaginé des langages a priori indépendants de la structure de la machine, assez proches des langages usuels de l'homme, les langages évolués.

L'objet de ce chapitre est de présenter d'une part une description du fonctionnement de ces langages pour chacun des niveaux cités précédemment, et d'autre part un panorama de la plupart des langages de programmation évolués.

Il ne faut surtout pas confondre apprentissage de la programmation et apprentissage d'un langage : le langage n'est qu'un moyen pour exprimer un algorithme compréhensible par la machine ; l'acte de programmer est d'abord un acte de conception d'algorithme, puis un travail de traduction dans un langage de programmation.

7.1 LE LANGAGE MACHINE

Le seul langage exécutable par un ordinateur est constitué d'instructions élémentaires codées en binaire ayant la structure définie précédemment : code opération, mode d'adressage, valeur d'adresse. Ce langage s'appelle le ***langage machine interne.***

Dans la mesure où les codes opérations correspondent à un circuit de calcul de l'unité arithmétique et logique, le jeu d'instructions est spécifique de chaque machine et il y a donc un langage machine interne exécutable par type d'ordinateur. En fait, l'introduction massive des microprocesseurs dans les petites machines conduit à une relative standardisation du langage machine, ce qui permet de disposer éventuellement d'outils compatibles.

7.2 LE LANGAGE D'ASSEMBLAGE

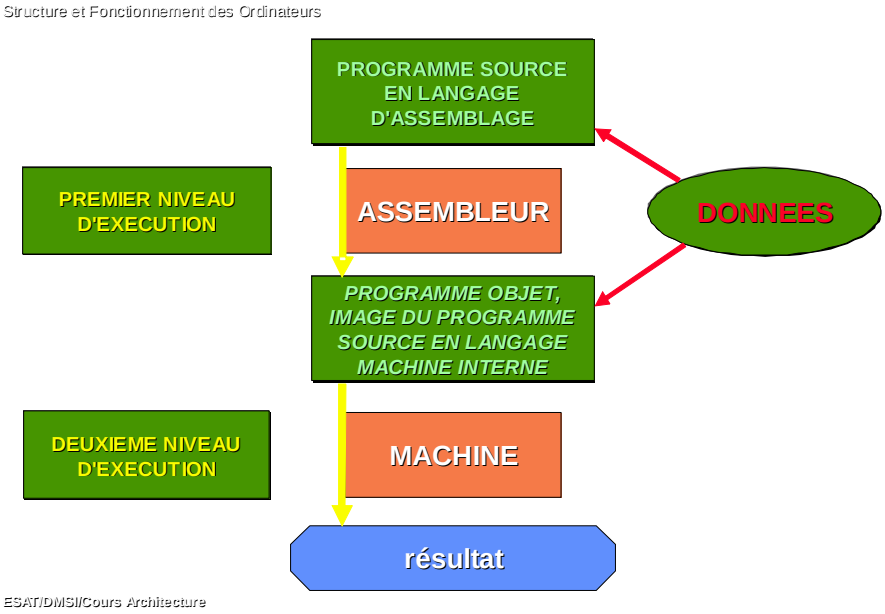
Le ***langage d'assemblage*** permet de se dégager de la gestion de la mémoire au sens où ce mot a été utilisé dans les exercices précédents. Le programmeur peut ainsi nommer des variables qu'il utilise en recourant à des identificateurs de variables qui n'ont plus rien à voir avec leurs adresses d'implantation physique.

Par exemple, si dans un programme une variable désigne une longueur, sous certaines réserves, elle pourra être nommée par l'identificateur de variable *LONGUEUR*. En utilisant notre machine fictive au lieu d'écrire *ADD 36,* le programmeur écrira *ADD LONGUEUR.*

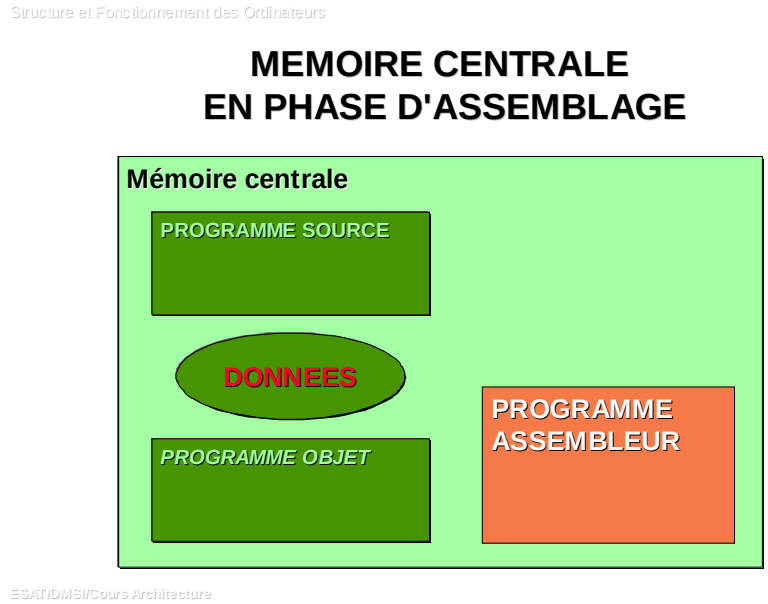
Pour que ce soit possible, il faut disposer d'un programme de traduction, l'***assembleur***, capable de transformer un programme écrit en langage d'assemblage en un programme équivalent au précédent transformé en langage machine interne directement exécutable par ordinateur.

De ce fait, le programmeur conçoit et écrit un programme en langage d'assemblage : c'est le ***programme source***. Ce programme source est considéré comme une donnée lors de l'exécution de l'assembleur, qui le traduit et génère un ***programme objet*** exprimé en langage machine interne, équivalent, du point de vue algorithmique, au programme source.

Ce programme objet peut alors être exécuté pour obtenir les résultats cherchés à partir de données de départ. Le schéma suivant résume ces deux niveaux d'exécution.



Après avoir introduit en mémoire centrale l'assembleur et le programme source, on peut lancer l'exécution de l'assembleur qui fabrique le programme objet. On a donc en mémoire centrale durant la phase d'assemblage les trois programmes, comme l'illustre le schéma suivant :



Du point de vue théorique, la phase d'assemblage est fondée sur les principes présentés précédemment. En réalité, il existe plusieurs types d'assembleurs :

- Ceux qui font le travail qui leur est assigné en une seule fois, un seul passage, ce sont les assembleurs à une passe.
- Ceux qui font le travail demandé en deux fois, deux passages, ce sont les assembleurs à deux passes.

De façon schématique, sans entrer dans les détails de réalisation et d'un point de vue purement fonctionnel, l'assembleur doit commencer par analyser chaque instruction du programme source. Il vérifie que le code opération correspond bien à un code connu qu'il peut retrouver dans une table des codes opérations dont il dispose et qui lui permet de connaître la valeur binaire correspondant à ce code opération.

Il vérifie que l'identificateur de variable figurant dans l'instruction respecte bien les règles définies pour le langage d'assemblage, comme par exemple le nombre de caractères utilisés dans l'identificateur, le type de ses caractères etc.. .

Il se construit une table des identificateurs de variables : soit cet identificateur existe déjà et il suffit de lui affecter l'adresse retenue pour la donnée, soit l'identificateur n'existe pas encore et l'assembleur affecte l'adresse physique d'un mot à cet identificateur. Il peut alors fabriquer les instructions en langage machine interne.

Cela revient à dire que le rôle de l'assembleur est tout d'abord de vérifier si les instructions du programme source sont correctes du point de vue syntaxique, puis de fabriquer les instructions du programme objet, une fois définies les adresses de la mémoire centrale correspondant aux variables utilisées dans le programme source.

En fait, les possibilités d'un langage d'assemblage sont plus importantes.

Le langage d'assemblage est encore utilisé pour réaliser des logiciels chez les constructeurs d'ordinateur mais n'est presque plus utilisé dans les applications de gestion. Un utilisateur n'aura jamais à manipuler ce type de langage. Ecrire un programme en langage d'assemblage exige que le programmeur connaisse bien la structure de la machine sur laquelle il travaille.

7.3 LES LANGAGES DE PROGRAMMATION EVOLUES

Ces langages ont été conçus au fur et à mesure du développement de l'informatique et n'ont pas tous les mêmes spécificités.

D'un point de vue théorique, un programme écrit dans un langage évolué donné devrait pouvoir être indépendant de la machine sur laquelle il doit être exécuté.

Un langage de programmation évolué permet d'écrire des instructions longues et complexes très éloignées des structures de la machine et proches des habitudes humaines.

Si l'on reprend le calcul proposé en cours :

```
TAC 3
ADD 1          addition de 42 avec 63
TME 10     stockage provisoire du résultat
```

Dans la très grande majorité des langages évolués, ce calcul peut être écrit en une seule instruction. Par exemple :

En **FORTRAN,** cela s'écrit a=42+63
En **COBOL,** cela s'écrit COMPUTE A = 42+63
En **BASIC,** cela s'écrit LET A = 42+63

Un programme, dit programme source, écrit en langage évolué doit être traduit par un programme, compilateur ou interpréteur, qui construit un programme objet en langage machine interne exécutable, équivalent au programme source du point de vue algorithmique.

Autrement dit, comme pour le langage d'assemblage, il faut commencer par exécuter le programme de traduction, compilateur ou interpréteur, avant de pouvoir exécuter le programme objet pour obtenir les résultats recherchés à partir des données de départ.

7.3.1 LES DIFFERENTES ETAPES

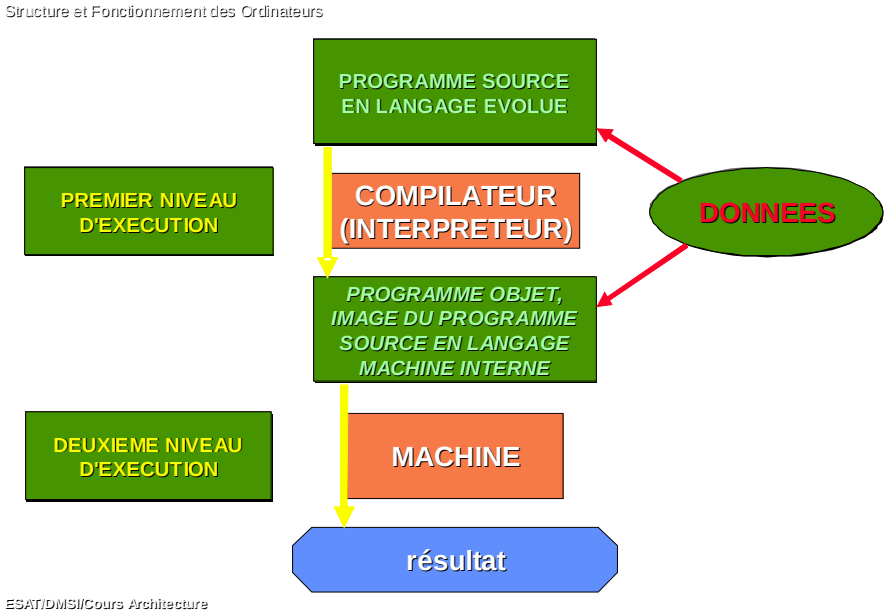
Mis à part le langage binaire, tous les langages nécessitent l'intervention d'outils spécifiques pour transformer le texte initial écrit par le programmeur (programme source) en programme exécutable (sous forme binaire).

Cette transformation s'opère en deux étapes : la ***compilation*** et ***l'édition de liens***.

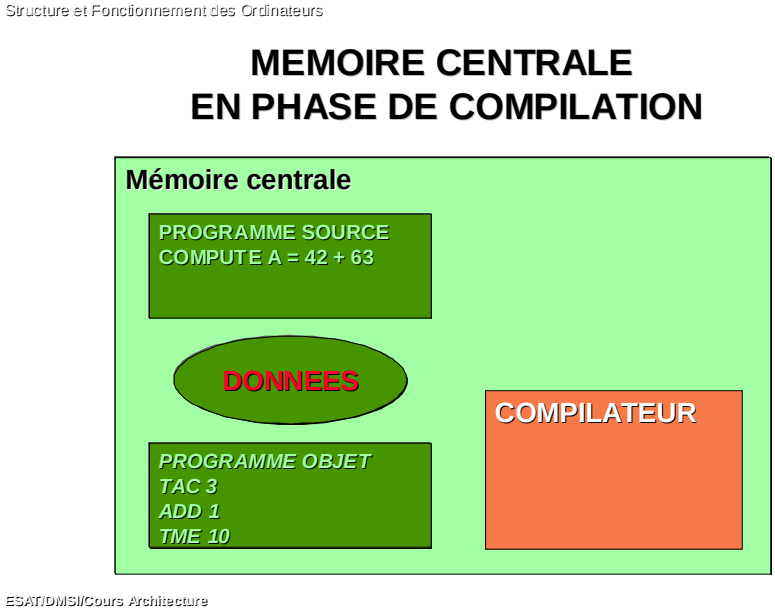
Le ***compilateur*** analyse le texte source puis signale les erreurs ou bien, dans le cas où il n'y pas d'erreur, génère un programme en langage binaire appelé programme objet.

La transformation du programme objet en programme exécutable s'opère grâce à l'édition de liens, qui permet d'associer au programme objet écrit par le programmeur d'autres programmes déjà compilés, soit également écrit par le programmeur, soit fournis avec le langage.

Comme précédemment, on peut constater deux niveaux d'exécution :



La mémoire centrale contient alors les éléments suivants :



Remarque : l'opération consistant à traduire un programme source écrit en langage d'assemblage en programme objet s'appelle assemblage et non compilation. L'outil correspondant est un assembleur et non un compilateur, cette opération étant plus simple.

7.3.2 FONCTIONNEMENT DE LA COMPILATION

La compilation permet de traduire le texte source en texte objet. Elle crée ainsi plusieurs modules (blocs) binaires indépendants correspondant chacun à une partie du programme ; on a en général un module par sous-programme. Cette compilation s'effectue en plusieurs étapes.

7.3.2.1 ANALYSE LEXICOGRAPHIQUE ET SEMANTIQUE

Le compilateur commence par vérifier "l'orthographe" et le vocabulaire du programme : noms des instructions, fautes de frappe,...

7.3.2.2 ANALYSE SYNTAXIQUE

Le compilateur contrôle ensuite la forme et la structure des lignes : compatibilité entre instructions et paramètres utilisés.

7.3.2.3 ANALYSE STRUCTURELLE

Le compilateur s'occupe ensuite de la cohérence interne du texte : structure globale, vérification des déclarations par rapport à leur utilisation...

7.3.2.4 TRADUCTION

Enfin, si aucune erreur n'a été détectée pendant les trois premières phases, le compilateur traduit chaque ligne de source en binaire afin d'obtenir le programme objet. Certains compilateurs

traduisent le texte source en langage d'assemblage, puis font appel à un assembleur qui lui effectue la traduction en binaire.

7.3.3 FONCTIONNEMENT DE L'EDITEUR DE LIEN (LINK)

L'éditeur de liens permet de passer du programme objet au programme exécutable. Pour cela il lie au programme déjà compilé différentes bibliothèques fournies soit par le concepteur du compilateur (bibliothèques système) soit déjà réalisées par le programmeur.

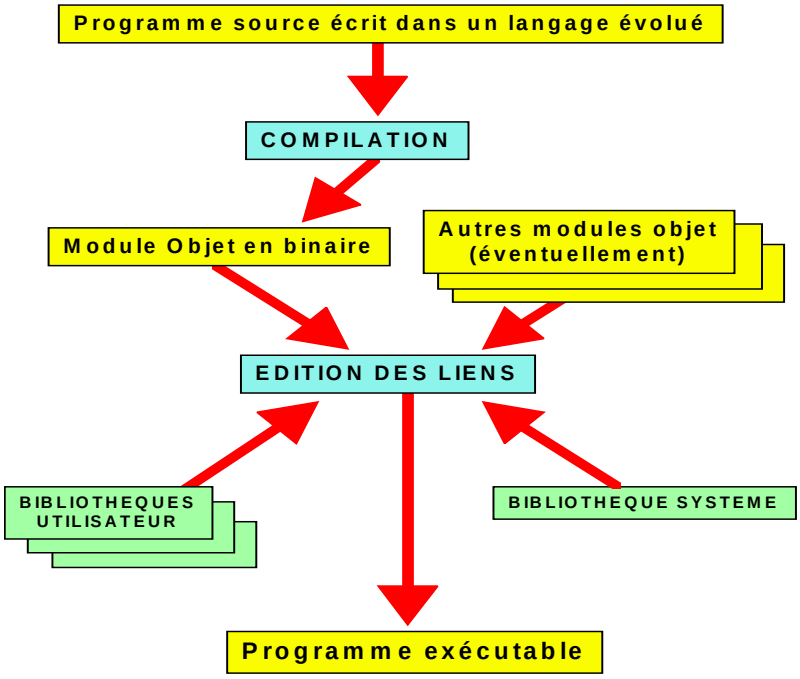
Un programme est constitué de plusieurs blocs qui s'appellent les uns les autres. Ces blocs sont parfois d'origine externe (bibliothèques système, bibliothèques graphiques) et se présentent sous forme binaire.

A la compilation du module principal, le compilateur rajoute en tête de celui-ci une table de références externes qui indique quels sous-programmes sont appelés, ainsi que les adresses de ces appels. A la compilation des bibliothèques, le compilateur rajoute une table précisant les adresses et tailles des routines.

L'éditeur de liens incorpore ensuite les modules utilisés par le module principal dans le corps du programme, puis initialise les adresses d'appel contenues dans le module principal avec les adresses d'implantation des routines de bibliothèque, au moyen de la table des références externes.

On obtient alors un module complet qui constitue le programme exécutable, et qui sera chargé en mémoire par le système d'exploitation lors de l'exécution.

Schéma de synthèse : compilation et édition des liens.



7.3.4 L'INTERPRETEUR

Contrairement à ce qui précède, l'interpréteur traduit une par une les instructions du programme source en langage évolué : il les analyse, les vérifie et produit immédiatement les instructions machine correspondantes. Chaque fois que l'on reprend un programme source, l'interpréteur doit réexécuter l'ensemble de son travail.

Un interpréteur est très agréable à utiliser dans la mesure où il signale les erreurs de syntaxe à chaque instruction et que le programmeur peut les corriger immédiatement.

Un compilateur est moins facile à utiliser : s'il détecte une erreur de syntaxe à un endroit, il la signale et, chaque fois qu'il la retrouve, il la signale à nouveau ; de plus, une erreur peut en entraîner une multitude d'autres qui n'en sont que la conséquence et qui disparaîtront lorsque la première sera corrigée. Par contre, un compilateur est plus complet qu'un interpréteur dans la mesure où il est capable de faire des recoupements entre différentes parties d'un programme.

7.4 CLASSIFICATION DES LANGAGES

Les langages informatiques peuvent être regroupés en différentes générations selon la proximité avec la machine.

Première génération

Il s'agit du **langage machine**, seul langage directement compréhensible par le processeur. C'est un langage binaire (suite de 0 et de 1), donc très fastidieux à écrire. Une légère simplification consistera à regrouper les bits en octets et à utiliser la notation hexadécimale.

On classe les codes obtenus en "codes opération" qui correspondent chacun à une action précise au niveau du processeur.

Deuxième génération

Elle correspond au **langage d'assemblage** (appelé couramment assembleur). Celui-ci simplifie la tâche de l'utilisateur en faisant correspondre à chaque catégorie de codes opération un mot appelé **mnémonique** (exemple : MOVE pour placer une valeur dans une case mémoire).

Cependant, il ne s'agit que d'une traduction facilitée en langage machine, et par conséquent le langage d'assemblage est particulier à un processeur donné ce qui implique de connaître parfaitement l'architecture de la machine de travail.

Troisième génération

Cette catégorie comprend tous les langages dits "évolués". Ces langages ont pour objectif de réduire les inconvénients du langage d'assemblage, ils doivent :

- **Etre indépendant du processeur** et donc pouvoir être utilisé sur différents types de machines (après une génération de programme exécutable par des outils adaptés à la machine).
- **Etre facile à apprendre**, et le plus proche possible du langage naturel.
- **Posséder le plus de fonctions de haut niveau possible** (édition, structures de contrôle, fichiers.).

Quatrième génération

Elle englobe (cela varie selon les auteurs) :

- Les langages spécifiques des applicatifs orientés vers des actions précises (exemple : langage DBASE pour la création de bases de données).

- Les langages objets (SMALLTALK) ou orientés objets (C++, EIFFEL) qui redéfinissent la façon de concevoir une application.

Cinquième génération

On y regroupe les langages dédiés à l'intelligence artificielle (LISP, PROLOG).

7.5 LES LANGAGES LES PLUS CONNUS

FORTRAN
L'objet du langage FORTRAN est de simplifier l'expression de calculs numériques en facilitant la résolution de problèmes comportant de nombreuses formules et variables. Il reste le langage le plus utilisé dans les applications scientifiques. La plupart des bibliothèques de programmes scientifiques implantées sur machine sont écrites dans ce langage.

COBOL
Le COBOL (Common Business Oriented Language) est un langage orienté vers la gestion de fichiers, défini à la demande d'un groupe d'utilisateurs, d'universitaires et de constructeurs le CODASYL (Committee On DAta System anaLysis), dont le rôle reste essentiel dans le domaine de la définition des structures de données. Une des principales contributions du langage COBOL a été d'offrir aux programmeurs la possibilité d'écrire des programmes, en employant des expressions tirées des langages naturels, il accorde à la description des données une place aussi importante que celle réservée aux instructions.

ALGOL
L'ALGOL (ALGorithmic Oriented Language) est conçu en 1958 par un groupe de travail européen. Il permet d'exprimer correctement tous les algorithmes. Il permet de travailler en respectant les structures algorithmiques. De plus ALGOL permet la récursivité, une procédure algol peut s'appeler elle-même.

BASIC
Conçu en 1965 par un groupe d'enseignants et d'étudiants le BASIC (Beginner's All-purpose Symbolic Instruction Code) était destiné à l'enseignement de la programmation pour des utilisateurs débutants.
Travaillant avec un interpréteur, il est aisé à apprendre puisque quelques heures suffisent pour pouvoir réaliser des programmes simples.

PASCAL
Le langage PASCAL, dont le nom fait référence au philosophe et mathématicien français, a été conçu au début des années 70 par une équipe de chercheurs suisses.
Sa syntaxe permet d'appliquer les principes de l'algorithmique, les principes d'organisation modulaire des programmes et les principes de structuration des données.

C
Au départ langage proche de la machine, développé par des universitaires, le C (ANSI et toutes ses variantes) est un des langages standard.

JAVA
Initialement développé par la société Sun Microsystems, le langage JAVA est un langage de programmation multi-plateformes et multi-systèmes.

